



Hybrid Obfuscated Key Exchange and KEMs

Felix Günther¹, Michael Rosenberg², Douglas Stebila³,
and Shannon Veitch⁴

¹ IBM Research Europe – Zurich, Rüschlikon, Switzerland
`mail@felixguenther.info`

² Cloudflare – New York, New York, USA
`research@mrosenberg.pub`

³ University of Waterloo, Waterloo, ON, Canada
`dstebila@uwaterloo.ca`

⁴ Department of Computer Science, ETH Zurich, Zurich, Switzerland
`shannon.veitch@inf.ethz.ch`

Abstract. Hiding the metadata in Internet protocols serves to protect user privacy, dissuade traffic analysis, and prevent network ossification. *Fully encrypted protocols* require even the initial key exchange to be obfuscated: a passive observer should be unable to distinguish a protocol execution from an exchange of random bitstrings. Deployed obfuscated key exchanges such as Tor’s pluggable transport protocol **obfs4** are Diffie–Hellman-based, and rely on the Elligator encoding for obfuscation. Recently, Günther, Stebila, and Veitch (CCS ’24) proposed a post-quantum variant **pq-obfs**, using a novel building block called *obfuscated key encapsulation mechanisms* (OKEMs): KEMs whose public keys and ciphertexts look like random bitstrings.

For transitioning real-world protocols, pure post-quantum security is not enough. Many are taking a *hybrid* approach, combining traditional and post-quantum schemes to hedge against security failures in either component. While hybrid KEMs are already widely deployed (e.g., in TLS 1.3), existing hybridization techniques fail to provide hybrid obfuscation guarantees for OKEMs. Further, even if a hybrid OKEM existed, the **pq-obfs** protocol would still not achieve hybrid obfuscation.

In this work, we address these challenges by presenting the first OKEM combiner that achieves hybrid IND-CCA security with hybrid ciphertext obfuscation guarantees, and using this to build **Drive1**, a modification of **pq-obfs** that is compatible with hybrid OKEMs. Our OKEM combiner allows for a variety of practical instantiations, e.g., combining obfuscated versions of DHKEM and ML-KEM. We additionally provide techniques to achieve unconditional public key obfuscation for LWE-based OKEMs, and explore broader applications of hybrid OKEMs, including a construction of the first hybrid password-authenticated key exchange (PAKE) protocol secure against adaptive corruptions in the UC model.

Keywords: KEM combiners · hybrid · obfuscation · anonymity · key exchange · ML-KEM · quantum-safe

1 Introduction

Increasingly, Internet protocols are hiding not just message contents, but the protocol metadata itself, due to metadata being repeatedly leveraged to violate user security and privacy, and to degrade connectivity [5, 37, 63, 66]. As a pertinent example, network censors often block traffic based on certain identifiable (plaintext) features of the protocol [64]. To evade this form of censorship, circumvention tools often encrypt metadata to avoid protocol fingerprinting and blocklists [1, 2, 58]. These *fully encrypted protocols* [27], such as the **obfs4** protocol integrated in the Tor ecosystem [58], produce a stream of random-looking bytes intended to avoid classification. Such streams can also be used in steganography due to their lack of structure [60]. Another motivation for encrypting protocol metadata, e.g., in pseudorandom cTLS [57], is to prevent *network ossification*, whereby network middleboxes expect a specific protocol mode (such as TLS 1.2) and fail to parse and route traffic of newer protocol versions (such as TLS 1.3 without its intentional 1.2 compatibility). If protocol metadata appears to a middlebox as a random string of bytes, then no parsers can be written in the first place. Random-looking metadata as an anti-ossification tool is already widely deployed, appearing in the QUIC protocol [41], where elements of the initial packet are encrypted to make the wire image of the protocol closer to pseudorandom, and in TLS Encrypted Client Hello [55], where the server “accept” response is encoded as a truncated MAC and hidden in the randomness sent from server to client. Both these techniques are also employed to improve privacy guarantees.

Recent formalizations capture the guarantees provided by fully encrypted protocols in the context of censorship circumvention [27, 34]. A shared secret key allows encrypting (meta)data into random-looking strings of bytes. However, it is less obvious how to make the initial key exchange of a connection look random—this is what *obfuscated key exchange* protocols aim to achieve.

1.1 Post-quantum Obfuscated Key Exchange and Beyond

As timelines for cryptographically relevant quantum computers shorten, there is an increased need to update current obfuscated key exchange protocols which derive their security from elliptic curve Diffie–Hellman. Recent work [34] proposes a quantum-safe variant of **obfs4** called **pq-obfs**. The construction relies on the newly defined *obfuscated key encapsulation mechanism* (OKEM): a KEM that supports random-looking encoding of public keys and ciphertexts. That work shows how to construct an efficient OKEM from the Module Learning with Errors (MLWE) assumption by introducing the **Kameleon** encoding which—akin to the **Elligator2** encoding for elliptic curve Diffie–Hellman [18]—turns ML-KEM public keys and ciphertexts into random strings.

Due to caution over switching to newer assumptions, some governments [28, 30] and cloud providers [61] opt instead for *hybrid* constructions, i.e., constructions whose assumptions combine the hardness of two input primitives, usually one post-quantum and one well-established. The reasons for this approach

are twofold. First, using two hardness assumptions hedges risk of mathematical advances, since the resulting construction is secure even if one of the underlying assumptions fails, as happened with NIST PQC Round 3 alternates and a finalist [10, 19, 23]. Second, it hedges risk of implementation errors, since newer post-quantum implementations may be more prone to bugs [17, 35, 36, 40]. For these reasons, hybrid protocols, such as the X25519MLKEM768 key agreement protocol for TLS 1.3 have already been deployed to millions worldwide [53]. It would thus be ideal to migrate obfuscated key exchange protocols to post-quantum using a hybrid solution rather than a purely lattice-based one.

To date, the work on building hybrid KEMs and key exchange has focused primarily on IND-CCA security [12, 20, 31, 62]. And although it has already been argued that a broader study of security properties should be considered for post-quantum KEMs [32], works exploring additional properties in the hybrid setting are very sparse [29]. To this end, we initiate a study of obfuscation and anonymity guarantees in the hybrid setting, towards constructing quantum-safe metadata-hiding protocols that are in line with modern techniques of hybridization.

1.2 Barriers to Hybrid Obfuscated Key Exchange

One might think that a straightforward approach to construct hybrid obfuscated key exchange is to instantiate `pq-obfs` with a hybrid OKEM using a standard KEM combiner. This however does not work, for two reasons.

First, there are no OKEM-specific hybridization methods, and the existing methods for KEM hybridization do not produce hybrid OKEMs. A KEM combiner takes as input two KEMs and produces a single, hybrid KEM whose security is maintained as long as one of the two input KEMs remains secure. Existing combiner techniques [12, 24, 31, 62] do not apply to the obfuscation properties of OKEMs, where the goal is to have public keys and ciphertexts indistinguishable from random strings. Take for example the *Xyber* hybrid KEM [62], which encapsulates by performing a *Kyber* encapsulation and X25519 key agreement in parallel, and where the final shared secret is a key derived from the two resulting shared secrets. While parallel combiners like this provide hybrid IND-CCA security, they do not provide hybrid obfuscation for OKEMs: if an adversary receives two concatenated public keys or ciphertexts, then they need only observe a non-uniformity in *one* of them to determine that the joint public key/ciphertext is not random. Most existing combiners are in this parallel style, depicted in Fig. 1(a), which even when applied to OKEMs only provides obfuscation as strong as the *weakest* of the underlying obfuscation assumptions.

Second, even if there were an OKEM hybridization technique, there is still a hurdle in using it to make `pq-obfs` hybrid. `pq-obfs` requires its OKEM to have *public key uniformity*: public keys must appear uniformly random. Since a KEM public key must be available for encapsulation prior to any other communication, it appears the only way to represent the combined public key is as the concatenation of the underlying public keys, or something equivalent. As a consequence, constructing a combined OKEM with hybrid public key uniformity seems to require that the two underlying OKEMs have unconditional public

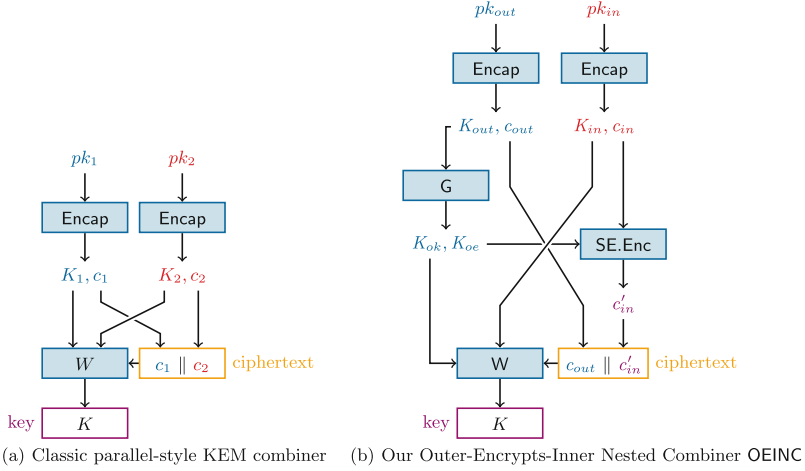


Fig. 1. Structure of Encap in KEM Combiners. W is a split-key PRF, G is a pseudo-random generator, and SE is symmetric encryption.

key uniformity. Unconditional public key uniformity, however, is uncommon for post-quantum (O)KEMs, where public keys have structure that becomes distinguishable from random if the underlying hardness assumption breaks down. This bars the path to make **pq-obfs** into a hybrid obfuscated key exchange using standard components.

1.3 Contributions

We discuss how we overcome these limitations to achieve hybrid obfuscated key exchange and hybrid password-authenticated key exchange protocols.

OKEM Combiner. Our first contribution is OEINC (“oink”), our *Outer-Encrypts-Inner Nested Combiner* for OKEMs which yields hybrid IND-CCA security and ciphertext obfuscation (without public key obfuscation) properties. The combiner is not fully generic, as it requires one of the component OKEMs to have statistically uniform ciphertexts. This allows us to use a nested approach in which we leverage the stronger OKEM as the *outer* one, using its shared secret towards encrypting the ciphertext of the weaker, *inner* OKEM. Our nested approach is depicted in Fig. 1(b). This provides hybrid guarantees for ciphertext uniformity and strong pseudorandomness.

We show that the necessary statistical uniformity of ciphertexts can be achieved using the DHKEM construction from HPKE [13], over a prime-order curve such as P-256 [52], with the Elligator² [59] encoding algorithm, or HPKE over X25519 or Ristretto [46] with the Elligator2 [18] encoding algorithm. This obfuscated variant of DHKEM can then be combined with a post-quantum OKEM, such as ML-Kameleon [34], to achieve the desired hybrid guarantees. We use OEINC as a building block for the following applications.

Application: Hybrid Obfuscated Key Exchange. Naturally, we would like to apply our OKEM combiner to obtain a hybrid obfuscated key exchange protocol, but, as noted above, hybrid `pq-obfs` would require hybrid public key uniformity, which OEINC does not achieve and generally seems elusive for hybrid OKEMs with post-quantum security.

We present `Drivel`, a modification to the `pq-obfs` protocol which permits the use of OKEMs *without* needing to obfuscate public keys. Our key insight is that public keys need not be sent in the clear, but can instead be encrypted with intermediate secrets in the key exchange. We describe the new protocol in detail and prove that `Drivel` achieves the same security guarantees as `pq-obfs`.

Application: Hybrid PAKE with Adaptive Corruptions. Finally, we consider how to apply our combiner to construct a hybrid password-authenticated key exchange (PAKE) protocol. Existing hybrid PAKEs are secure only in the static corruption model, where the adversary may not corrupt any parties during the protocol execution [39, 48]. We construct an adaptively secure hybrid PAKE.

We observe that `CAKE` [14], which is proven secure in the UC model [22] with adaptive corruptions, can be instantiated with any OKEM with ciphertext and public key uniformity. As previously noted, public key uniformity guarantees for combiners seem elusive unless both underlying OKEMs have unconditional public key uniformity. We describe a method to achieve unconditional public key uniformity in *LWE*-based OKEMs. We observe that some standards-track KEMs, such as `FrodoKEM` [21, 51], have a *dual-LWE* structure: both public keys and ciphertexts are *LWE* samples. We show how removing this optimization (reverting back to earlier ideas of [54]) yields a statistically uniform public key, at the expense of a $15\times$ increase in ciphertext size.

Combining this *LWE*-based OKEM with a classical OKEM with unconditional ciphertext uniformity via OEINC, and then using it in `CAKE`, we achieve the first known hybrid PAKE that is secure under adaptive corruptions.

2 Preliminaries

We write $y \leftarrow A(z)$ to denote assigning y the output of a deterministic algorithm $A(z)$. When A is non-deterministic, we write $y \leftarrow_s A(z)$. Similarly, we write $y \leftarrow_s S$ to denote uniformly sampling an element from the set S .

A *pseudorandom function* (PRF) $F: \mathcal{X} \times \mathcal{Y} \rightarrow \mathcal{Z}$ treats the first input as a key and the second as a label. For a randomly chosen key, the outputs on an adversarially-chosen label of F should be indistinguishable from those of a random function on the label input. The *swap* of F is $F'(y, x) = F(x, y)$. We say that F is a *swap-PRF* if its swap F' is a PRF. A *dual-PRF* is both a PRF and a swap-PRF. We defer the standard definitions of PRF security, pseudorandom generator (PRG) security, and OT-IND\$ security (one-time indistinguishability of ciphertexts from random) of symmetric encryption to the full version [33].

We adopt the definition of a *split-key pseudorandom function* from [31], which is a generalization of a pseudorandom function to multiple key inputs

that behaves like a random function if at least one of its key inputs is picked uniformly at random.

Definition 1 (Split-key pseudorandom function). A split-key pseudorandom function $F: \mathcal{K}_1 \times \cdots \times \mathcal{K}_n \times \mathcal{X} \rightarrow \mathcal{Y}$ takes as input a finite number of keys in $\mathcal{K}_1 \times \cdots \times \mathcal{K}_n$ and a label in $\mathcal{X}$ and produces an output in $\mathcal{Y}$. We define the split-key pseudorandom function (skPRF) advantage of an adversary $\mathcal{A}$ against a function F as

$$\begin{aligned} \text{Adv}_{F,i}^{\text{skPRF}}(\mathcal{A}) := & \Pr \left[\mathcal{A}^{F(\cdots, k_i, \cdots)}() \Rightarrow 1 \mid k_i \leftarrow \mathcal{K}_i \right] \\ & - \Pr \left[\mathcal{A}^{R(\cdots)}() \Rightarrow 1 \mid R \leftarrow \mathcal{R} \{ \text{all functions } : \mathcal{K}_{n \setminus i} \times \mathcal{X} \rightarrow \mathcal{Y} \} \right], \end{aligned}$$

where $\mathcal{K}_{n \setminus i}$ denotes $\mathcal{K}_1 \times \cdots \times \mathcal{K}_{i-1} \times \mathcal{K}_{i+1} \times \cdots \times \mathcal{K}_n$.

2.1 Obfuscated KEMs

Günther, Stebila, and Veitch [34] introduced notions of obfuscated key encapsulation mechanisms (OKEMs). We present equivalent definitions next, with slight modifications for simplicity.¹

Definition 2 (Key encapsulation mechanism). A key encapsulation mechanism $\text{KEM} = (\text{KGen}, \text{Encap}, \text{Decap})$ consists of three algorithms:

- $\text{KGen}() \xrightarrow{s} (sk, pk)$ is a probabilistic key generation algorithm that generates a secret key sk and corresponding public key pk .
- $\text{Encap}(pk) \xrightarrow{s} (c, K)$ is a probabilistic encapsulation algorithm that takes as input a KEM public key pk , and outputs a ciphertext c and shared secret K .
- $\text{Decap}(sk, c) \rightarrow K$ is a deterministic decapsulation algorithm that takes as input a secret key sk and ciphertext c , and outputs a shared secret K .

Definition 3 (KEM encapsulation/decapsulation correctness). We say that a KEM $\text{KEM} = (\text{KGen}, \text{Encap}, \text{Decap})$ is δ_{KEM} -correct if

$$\Pr \left[\text{Decap}(sk, c) \neq K \mid \begin{array}{l} (sk, pk) \leftarrow \mathcal{KGen}(), \\ (c, K) \leftarrow \mathcal{Encap}(pk) \end{array} \right] \leq \delta_{\text{KEM}}.$$

IND-CCA/CPA and SPR-CCA/CPA security. We will consider two security flavors for KEMs: (1) indistinguishability under chosen-ciphertext or chosen-plaintext attacks (IND-CCA / IND-CPA), capturing that the shared secret is

¹ Concretely, we integrate the encoding/decoding steps into the key generation, encapsulation, and decapsulation algorithms rather than having separate encoding/decoding algorithms. This is because in some cases, e.g., when Kemeleon is used with rejection sampling, not every key/ciphertext can be encoded. This means that the failure rate of the encoding algorithm is captured directly in the security property of the OKEM, rather than as a separate property.

indistinguishable from random given a real ciphertext, and (2) strong pseudorandomness under chosen-ciphertext attacks (SPR-CCA) [65] as well as its chosen-plaintext version (SPR-CPA), capturing that a real (ciphertext, shared secret) pair is indistinguishable from a random pair. In SPR-CCA/SPR-CPA, pseudorandomness of the ciphertext is defined with respect to a simulator $\mathcal{S}$ defining a ciphertext target distribution. When the simulator $\mathcal{S}$ outputs uniformly random bitstrings, we write SPR-CCA/SPR-CPA.

We provide the formal security games in Fig. 2 and define the respective advantages of an adversary $\mathcal{A}$ against a KEM K and $\text{atk} \in \{\text{CPA}, \text{CCA}\}$ as

$$\begin{aligned}\text{Adv}_K^{\text{IND-atk}}(\mathcal{A}) &:= 2 \cdot \Pr \left[G_K^{\text{IND-atk}}(\mathcal{A}) \Rightarrow 1 \right] - 1 \quad \text{and} \\ \text{Adv}_{K,\mathcal{S}}^{\text{SPR-atk}}(\mathcal{A}) &:= 2 \cdot \Pr \left[G_{K,\mathcal{S}}^{\text{SPR-atk}}(\mathcal{A}) \Rightarrow 1 \right] - 1.\end{aligned}$$

Definition 4 (KEM public key collision probability). *Let KEM be a KEM. We define the public key collision probability of KEM for $n \in \mathbb{N}$ public keys as*

$$\text{pkcoll}_{\text{KEM}}(n) := \Pr \left[\begin{array}{c} pk_i = pk_j \\ \wedge i \neq j \end{array} \middle| \begin{array}{c} (sk_i, pk_i) \leftarrow^* \text{KEM.KGen}() \\ \text{for } i \in [1, n] \end{array} \right].$$

Definition 5 (Obfuscated KEM). *An obfuscated key encapsulation mechanism (OKEM) $\mathcal{O} = (\text{KGen}, \text{Encap}, \text{Decap}, \text{DecodePk})$ with obfuscated key length $\text{ol} \in \mathbb{N}$ and obfuscated ciphertext length $\text{cl} \in \mathbb{N}$ consists of the following algorithms:*

- $\text{KGen}() \mapsto (sk, pk, \widehat{pk})$ is a probabilistic key generation algorithm that generates a secret key sk , public key pk , and obfuscated public key $\widehat{pk} \in \{0, 1\}^{\text{ol}}$.
- $\text{Encap}(pk) \mapsto (c, K)$ is a probabilistic encapsulation algorithm that takes as input a KEM public key pk and outputs an (obfuscated) ciphertext $c \in \{0, 1\}^{\text{cl}}$ and key K .
- $\text{Decap}(sk, c) \rightarrow K$ is a deterministic decapsulation algorithm that takes as input a secret key sk and (obfuscated) ciphertext c , and outputs a key K .
- $\text{DecodePk}(\widehat{pk}) \rightarrow pk$ is a deterministic decoding algorithm that on input an obfuscated public key $\widehat{pk} \in \{0, 1\}^{\text{ol}}$ outputs a public key pk .

Note that the tuple of algorithms $(\text{KGen}, \text{Encap}, \text{Decap})$, when ignoring the obfuscated public key $\widehat{pk}$ output by KGen , is a KEM as in Definition 2.

Beyond KEM correctness, as per Definition 3, we demand that public keys generated by KGen can be successfully decoded:

$$\Pr \left[\text{DecodePk}(\widehat{pk}) = pk \mid (sk, pk, \widehat{pk}) \leftarrow^* \text{KGen}() \right] = 1.$$

IND-CPA, IND-CCA, SPR-CPA, and SPR-CCA security of an OKEM are defined exactly as for the underlying KEM (see Fig. 2), merely ignoring the

$G_K^{\text{IND-CCA}}(\mathcal{A})$:	$G_{K,S}^{\text{SPR-CCA}}(\mathcal{A})$:	$\mathcal{O}_{\text{Decap}}(c)$:
1 $b \leftarrow_{\$} \{0, 1\}$	7 $b \leftarrow_{\$} \{0, 1\}$	14 if $c = c^*$ then return $\perp$
2 $(pk, sk) \leftarrow_{\$} \text{KGen}()$	8 $(pk, sk) \leftarrow_{\$} \text{KGen}()$	15 $K \leftarrow \text{Decap}(sk, c)$
3 $(c^*, K_1^*) \leftarrow_{\$} \text{Encap}(pk)$	9 $(c_1^*, K_1^*) \leftarrow_{\$} \text{Encap}(pk)$	16 return K
4 $K_0^* \leftarrow_{\$} \mathcal{K}$	10 $c_0^* \leftarrow_{\$} \mathcal{S}; K_0^* \leftarrow_{\$} \mathcal{K}$	
5 $b' \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\text{Decap}}(\cdot)}(pk, c^*, K_b^*)$	11 $c^* \leftarrow c_b^*$	
6 return $\llbracket b = b' \rrbracket$	12 $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Decap}}(\cdot)}(pk, c_b^*, K_b^*)$	
	13 return $\llbracket b = b' \rrbracket$	

Fig. 2. Security games for IND-CCA and SPR-CCA security of a KEM $K = (\text{KGen}, \text{Encap}, \text{Decap})$ with key space $\mathcal{K}$. The IND-CPA and SPR-CPA games are obtained by removing the decapsulation oracle from IND-CCA and SPR-CCA, respectively. We write SPR $\mathcal{S}$ -CCA or SPR $\mathcal{S}$ -CPA when $\mathcal{S}$ outputs uniformly random bitstrings of some appropriate length cl .

encoded public key $\widehat{pk}$ output by KGen . We also adopt notions of public key and ciphertext uniformity of (O)KEMs from [34].²

Definition 6 (Public key uniformity). Let $\mathcal{O}$ be an OKEM. We measure the uniformity of the obfuscated public keys of length ol generated by $\mathcal{O}.\text{KGen}$ against an adversary $\mathcal{A}$ as

$$\text{Adv}_{\mathcal{O}}^{\text{pk-unif}}(\mathcal{A}) := 2 \cdot \Pr \left[\mathcal{A}(\widehat{pk}_b) = b \mid \begin{array}{l} b \leftarrow_{\$} \{0, 1\}, \widehat{pk}_0 \leftarrow_{\$} \{0, 1\}^{\text{ol}}, \\ (sk_1, pk_1, \widehat{pk}_1) \leftarrow_{\$} \mathcal{O}.\text{KGen}() \end{array} \right] - 1.$$

For an unbounded adversary $\mathcal{A}$, we call the advantage $\text{Adv}_{\mathcal{O}}^{\text{pk-unif}}(\mathcal{A})$ statistical.

Definition 7 (Ciphertext uniformity – strong or regular). Let $\mathcal{O}$ be an OKEM. We measure the strong or regular ciphertext uniformity of the obfuscated ciphertext of length cl generated by $\mathcal{O}.\text{Encap}$ against an adversary $\mathcal{A}$ as

$$\text{Adv}_{\mathcal{O}}^{\text{atk-ctxt-unif}}(\mathcal{A}) := 2 \cdot \Pr \left[\mathcal{A}(\overset{\text{---}}{\underset{\text{---}}{\text{sk}}}, \overset{\text{---}}{\underset{\text{---}}{pk}}, c_b) = b \mid \begin{array}{l} b \leftarrow_{\$} \{0, 1\}, c_0 \leftarrow_{\$} \{0, 1\}^{\text{cl}}, \\ (sk, pk, \widehat{pk}) \leftarrow_{\$} \mathcal{O}.\text{KGen}(), \\ (c_1, K_1) \leftarrow_{\$} \mathcal{O}.\text{Encap}(pk) \end{array} \right] - 1,$$

where $\text{atk} \in \{\text{strong}, \text{reg}\}$ and the code in the $\overset{\text{---}}{\underset{\text{---}}{\text{dashed box}}}$ is only included for $\text{atk} = \text{strong}$.

For an unbounded adversary $\mathcal{A}$, we call the advantage $\text{Adv}_{\mathcal{O}}^{\text{atk-ctxt-unif}}(\mathcal{A})$ statistical.

² Our **Drivel** protocol does not require public key uniformity. We still discuss the notion, as it shows up in the prior analysis of **obfs4** and **pq-obfs** [34], and for **PAKEs**.

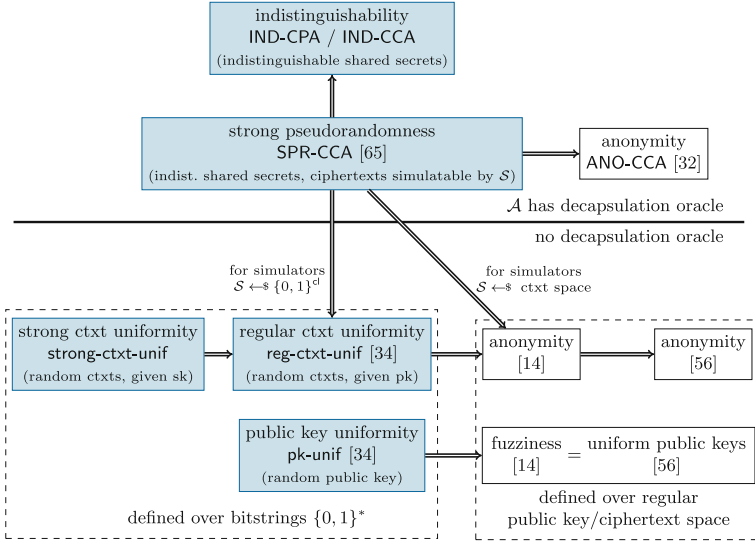


Fig. 3. Relations between anonymity and uniformity notions for KEMs. Arrows $\Rightarrow$ indicate strict implications. Properties shaded in blue are used in this paper.

We note that statistical regular ciphertext uniformity implies strong ciphertext uniformity, because a computationally unbounded adversary can simply enumerate all secret keys sk that produce the corresponding public key pk . Therefore, we simply write statistical ciphertext uniformity (without qualifier).

When paired with an encoding function like *Elligator*² [59], a Diffie–Hellman-based KEM over a prime-order curve (e.g., DHKEM (P-256) [13]) is an OKEM with statistical ciphertext uniformity and statistical public key uniformity (see the full version [33]). Additionally, ML-KEM paired with the Kameleon encoding is an OKEM with (computational) regular ciphertext uniformity and (computational) public key uniformity, as proven in [34, §2.4]. Finally, Saber [25] and FrodoKEM [21] have the same properties as ML-KEM with Kameleon and are “natural” OKEMs, i.e., KEMs where no encoding step is necessary in order to achieve obfuscation. The IND-CCA and SPR-CCA properties of Saber and FrodoKEM are stated in [32, 49, 65].³

The notions of strong pseudorandomness, public key uniformity, and (strong and regular) ciphertext uniformity are related to existing notions of anonymity and uniformity for KEMs in the literature, summarized in Fig. 3. We elaborate on these relations in the full version [33], and note that all related notions are weaker than or equivalent to one of the properties we use here. The stronger notions of anonymity and uniformity are required for our later applications (see Sects. 4 and 5). We adopt the above definitions of ciphertext and public key uniformity

³ Specifically, [32] shows that FrodoKEM achieves computational ANO-CCA, and [65] suggests that it also achieves computational SPR-CCA.

from [34] (rather than prior definitions of uniformity), because we are interested in uniformity properties defined over the space of bitstrings $\{0,1\}^*$ rather than the space of public keys/ciphertexts as used in some prior works. Although *regular* ciphertext uniformity is implied by SPR\$-CCA (SPR-CCA defined with respect to a simulator that outputs uniformly random bitstrings), we consider it separately because in some results we only require the weaker assumption.

Finally, to prove PAKE security, we include the following definition, similar but not equivalent to `ctxt-unif`. We rephrase the original in terms of OKEM.

Definition 8 (Strong second pseudorandomness [42, Def. 2.5]). *Let $\mathcal{O}$ be an OKEM with obfuscated ciphertexts of length cl . We measure the strong second pseudorandomness of $\mathcal{O}$ against an adversary $\mathcal{A}$ as*

$$\text{Adv}_{\mathcal{O}}^{\text{strong-2nd-pr}}(\mathcal{A}) := 2 \cdot \Pr \left[\mathcal{A}(pk, c_b, K) = b \mid \begin{array}{l} b \leftarrow_{\$} \{0,1\}, \\ (sk, pk, \widehat{pk}) \leftarrow_{\$} \mathcal{O}.\text{KGen}(), \\ c_0 \leftarrow_{\$} \{0,1\}^{\text{cl}}, \\ c_1 \leftarrow_{\$} \mathcal{O}.\text{Encap}(pk), \\ K \leftarrow \text{Decap}(sk, c_b) \end{array} \right] - 1.$$

For an unbounded adversary $\mathcal{A}$, we call the advantage $\text{Adv}_{\mathcal{O}}^{\text{strong-2nd-pr}}(\mathcal{A})$ statistical.

3 OEINC: An OKEM Combiner

We now present our OKEM combiner and prove the necessary security properties for use in our hybrid obfuscated key exchange protocol (Sect. 4).

Shortcomings of Existing Combiners. A KEM combiner merges two ingredient KEMs into a single, hybrid KEM such that security of the hybrid is maintained as long as one of the ingredient KEMs remains secure. The main focus of prior work analyzing KEM combinators [20, 31] has been on achieving hybrid IND-CCA security: an attacker should not be able to learn the secret key by breaking only one of the underlying KEMs. Although this is a natural starting point, some applications such as broadcast encryption, anonymous credential systems, and auction protocols, require anonymity properties not implied by IND-CCA security [32]. Likewise, for our use case of constructing a hybrid obfuscated key exchange, we require that the combined KEM has *hybrid obfuscation* properties, to ensure that an adversary cannot distinguish the key exchange transcript from random by breaking obfuscation of only one of the underlying primitives.

Prior work on KEM combinators constructed *parallel combinars*: the combined public keys, secret keys, and ciphertexts are the concatenation of the ingredient KEM's public keys, secret keys, and ciphertexts, respectively (perhaps with an added MAC). The combined KEM's encapsulation and decapsulation routines first execute the underlying KEM's encapsulation and decapsulation routines and then join the resulting shared secrets using a *split-key PRF* [31] to derive the final shared secret. Figure 1(a) illustrates a parallel combiner.

These existing combiners, however, do not provide hybrid obfuscation. Specifically, if an ingredient KEM's ciphertext is distinguishable from random, then its concatenation with another ciphertext is trivially distinguishable from random as well. For example, since the uniformity of ML-KEM ciphertexts relies on the MLWE assumption, breaking this assumption would be sufficient to violate the ciphertext uniformity of any parallel combiner using ML-KEM, such as Xyber [62] or X-Wing [12]. Furthermore, it is not obvious how to fix this: if one were to encrypt each component ciphertext under a key from the respective other ingredient KEM, the result is impossible to decapsulate. Therefore, we require a more thoughtful construction.

The OEINC Construction. Our OKEM combiner's encapsulation function operates sequentially, rather than in parallel. This new method, depicted in Fig. 1(b), operates as follows. The first (*outer*) ingredient OKEM is run, and (a key derived from) its shared secret is used to encrypt the second (*inner*) ingredient OKEM's ciphertext. The final ciphertext is the outer ciphertext concatenated with the encrypted inner ciphertext. The shared secrets are combined with a split-key PRF. We call this the *Outer-Encrypts-Inner Nested Combiner* (OEINC).

The ciphertext uniformity of this new scheme relies on the security of the encryption scheme and the ciphertext uniformity of one of the two ingredient OKEMs. We use the OKEM with the stronger ciphertext uniformity property as the outer OKEM. When the outer OKEM has statistical ciphertext uniformity, e.g., as in DHKEM combined with Elligator2/Elligator² we find that the resulting combiner achieves hybrid security for the relevant notions.

Our detailed combiner construction is given in Fig. 4. Its building blocks include a length-preserving, OT-IND\$-secure symmetric encryption scheme SE with key space $\mathcal{K} = \{0,1\}^{k_{\text{SE}}}$ and message length $\text{cl}_{\text{inOKEM}}$, a function $G: \{0,1\}^{kl} \rightarrow \{0,1\}^{kl+k_{\text{SE}}}$ assumed to be a pseudorandom generator, and a split-key PRF W .

3.1 Overview of Security

We now prove that OEINC satisfies IND-CPA, IND-CCA, SPR-CCA, ciphertext uniformity (ctxt-unif), and public key uniformity (pk-unif), given some properties of the underlying OKEMs. We show IND-CPA, IND-CCA, and ciphertext uniformity separately from SPR-CCA (even though they are implied by the latter, cf. Fig. 3), since the weaker notions are sufficient for some applications and can be argued from simpler assumptions. More specifically, for OEINC instantiated with an outer OKEM outOKEM and an inner OKEM inOKEM :

- IND-CPA/IND-CCA: If either outOKEM or inOKEM is IND-CPA/IND-CCA, then the combined OKEM is IND-CPA/IND-CCA.
- SPR-CCA: If either (1) outOKEM is SPR-CCA, or (2) inOKEM is SPR-CCA and outOKEM is strong-ctxt-unif, then the combined OKEM is SPR-CCA.

KGen(): $\begin{aligned} &1 \quad (sk_{out}, pk_{out}, \widehat{pk}_{out}) \leftarrow \$\text{outOKEM.KGen}() \\ &2 \quad (sk_{in}, pk_{in}, \widehat{pk}_{in}) \leftarrow \$\text{inOKEM.KGen}() \\ &3 \quad \text{return } ((sk_{out}, sk_{in}), (pk_{out}, pk_{in}), \widehat{pk}_{out} \parallel \widehat{pk}_{in}) \end{aligned}$	DecodePk(pk): $\begin{aligned} &12 \quad \widehat{pk}_{out} \parallel \widehat{pk}_{in} \leftarrow \widehat{pk} \\ &13 \quad pk_{out} \leftarrow \text{outOKEM.DecodePk}(\widehat{pk}_{out}) \\ &14 \quad pk_{in} \leftarrow \text{inOKEM.DecodePk}(\widehat{pk}_{in}) \\ &15 \quad \text{return } (pk_{out}, pk_{in}) \end{aligned}$
Encap(pk): $\begin{aligned} &4 \quad (pk_{out}, pk_{in}) \leftarrow pk \\ &5 \quad (K_{out}, c_{out}) \leftarrow \$\text{outOKEM.Encap}(pk_{out}) \\ &6 \quad (K_{oe}, K_{ok}) \leftarrow G(K_{out}) \quad // \text{ keys for enc. \& key deriv.} \\ &7 \quad (K_{in}, c_{in}) \leftarrow \$\text{inOKEM.Encap}(pk_{in}) \\ &8 \quad c'_{in} \leftarrow \text{SE.Enc}(K_{oe}, c_{in}) \\ &9 \quad c \leftarrow c_{out} \parallel c'_{in} \\ &10 \quad K \leftarrow W(K_{ok}, K_{in}, c) \\ &11 \quad \text{return } (K, c) \end{aligned}$	Decap(sk, c): $\begin{aligned} &16 \quad (sk_{out}, sk_{in}) \leftarrow sk \\ &17 \quad c_{out} \parallel c'_{in} \leftarrow c \quad // \text{ ciphertexts are fixed-length} \\ &18 \quad K_{out} \leftarrow \text{outOKEM.Decap}(sk_{out}, c_{out}) \\ &19 \quad (K_{oe}, K_{ok}) \leftarrow G(K_{out}) \\ &20 \quad c_{in} \leftarrow \text{SE.Dec}(K_{oe}, c'_{in}) \\ &21 \quad K_{in} \leftarrow \text{inOKEM.Decap}(sk_{in}, c_{in}) \\ &22 \quad K \leftarrow W(K_{ok}, K_{in}, c) \\ &23 \quad \text{return } K \end{aligned}$

Fig. 4. Our OKEM combiner OEINC[outOKEM, inOKEM, SE, G, W] for two OKEMs outOKEM and inOKEM, SE a length-preserving symmetric encryption scheme, G a PRG, and W a split-key PRF.

- Ciphertext uniformity: If either (1) outOKEM is reg-ctxt-unif and outOKEM is IND-CCA, or (2) outOKEM is reg-ctxt-unif and inOKEM is reg-ctxt-unif, then the combined OKEM is reg-ctxt-unif.
- Public key uniformity: If both outOKEM and inOKEM are pk-unif, then the combined OKEM is pk-unif.
- Strong second pseudorandomness: If either (1) outOKEM is strong-2nd-pr and outOKEM is IND-CPA, or (2) outOKEM is strong-2nd-pr and inOKEM is strong-2nd-pr, then the combined OKEM is strong-2nd-pr.

In particular, when outOKEM is statistically strong-ctxt-unif and statistically strong-2nd-pr, the combined OKEM has IND-CPA/CCA, SPR-CCA, reg-ctxt-unif, and strong-2nd-pr properties that are the stronger of the two underlying OKEMs.

Public key uniformity stands out because it requires *both* underlying OKEMs to be pk-unif. While this limitation appears to be inherent to the primitive (when only the public key is known, there is no key material with which to obfuscate it), it is not a deal breaker for deployment. Our key exchange protocol in Sect. 4 does not require public key uniformity at all, since it encrypts ephemeral public keys using intermediate secrets derived from static keys. And where public key uniformity is necessary, e.g., in password-authenticated key exchange in Sect. 5, we show that it is possible to construct lattice-based OKEMs with statistical public key uniformity. This comes at the cost of significantly larger public keys and ciphertexts compared to, e.g., ML-KEM with Kameleon.

Formally, we establish *hybrid* security when the outer OKEM has unconditional pseudorandomness properties by proving *two* advantage bounds for each targeted security notion: one that relies on the outer OKEM's security, and

another that relies on the inner OKEM's security. Jointly, these bounds establish the hybrid security claims stated informally above.

We defer the proofs of IND-CCA, IND-CPA, reg-ctxt-unif, strong-2nd-pr, and pk-unif to the full version [33].

3.2 IND-CPA and IND-CCA Security

Intuitively, IND-CPA and IND-CCA security of OEINC should naturally hold, since the final shared secret is derived from the two underlying shared secrets using a split-key PRF. Importantly, OEINC does not use the same secret for encrypting ciphertexts (K_{oe}) as for key derivation (K_{ok}). The straightforward proof follows similarly to the proof of IND-CCA security for existing KEM combiners [31], up to the step of applying PRG security of G .

Theorem 1 (IND-CPA / IND-CCA security of OEINC). *Let $\text{OEINC} = \text{OEINC}[\text{outOKEM}, \text{inOKEM}, \text{SE}, G, W]$ be a combined OKEM as defined in Fig. 4. Then for any adversary $\mathcal{A}$ against the IND-atk security of OEINC, for $\text{atk} \in \{\text{CPA}, \text{CCA}\}$, we give algorithms $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3, \mathcal{C}_1, \mathcal{C}_2$ such that*

$$\text{Adv}_{\text{OEINC}}^{\text{IND-atk}}(\mathcal{A}) \leq \text{Adv}_{\text{outOKEM}}^{\text{IND-atk}}(\mathcal{B}_1) + \text{Adv}_G^{\text{PRG}}(\mathcal{B}_2) + \text{Adv}_{W,1}^{\text{skPRF}}(\mathcal{B}_3),$$

and

$$\text{Adv}_{\text{OEINC}}^{\text{IND-atk}}(\mathcal{A}) \leq \text{Adv}_{\text{inOKEM}}^{\text{IND-atk}}(\mathcal{C}_1) + \text{Adv}_{W,2}^{\text{skPRF}}(\mathcal{C}_2).$$

3.3 SPR-CCA Security

Consider the case where the underlying computational assumption of inOKEM is broken. Then, assuming the SPR-CCA security of the outer OKEM holds, this implies that the outer ciphertext is uniformly random and the inner ciphertext is encrypted (using an OT-IND\$ symmetric encryption scheme) with a random key, and thus also pseudorandom. The final shared secret is also random since the split-key PRF W takes as input the shared secret of the outer OKEM.

Now, consider the case where the underlying computational assumption of outOKEM is broken, i.e., IND-CCA and SPR-CCA no longer hold (note that strong ciphertext uniformity cannot fail when it is statistical). Then the outer ciphertext is still uniformly random, and the inner one is a keyed permutation of a uniformly random value, and thus still random even though the encryption key may be compromised. The shared secret is also uniformly random, since the split-key PRF W still consumes the shared secret of the inner OKEM.

Recall that we use SPR\$-CCA to denote SPR-CCA security when the simulator outputs uniformly random ciphertexts of length corresponding to the OKEM's obfuscated ciphertext length cl .

Theorem 2 (SPR\$-CCA Security of OEINC). *Let $\text{OEINC} = \text{OEINC}[\text{outOKEM}, \text{inOKEM}, \text{SE}, G, W]$ be a combined OKEM as defined in Fig. 4, where SE is a length-preserving symmetric encryption scheme, G a PRG, and W a split-key*

PRF. Then OEINC has strong pseudorandomness under chosen-ciphertext attack (SPR-CCA) if either (I) the outer KEM outOKEM is SPR-CCA-secure, SE is OT-IND $\$$ -secure, G is a PRG, and W is a PRF, or (II) inOKEM is SPR $\$$ -CCA-secure, outOKEM has strong ciphertext uniformity, and W is a PRF.

More precisely, for any adversary $\mathcal{A}$ against the SPR-CCA security of OEINC, we give algorithms $\mathcal{B}_1$ – $\mathcal{B}_4$ and $\mathcal{C}_1$ – $\mathcal{C}_3$ such that

$$\begin{aligned} \text{Adv}_{\text{OEINC}, S_1}^{\text{SPR-CCA}}(\mathcal{A}) &\leq \text{Adv}_{\text{outOKEM}, S_{\text{outOKEM}}}^{\text{SPR-CCA}}(\mathcal{B}_1) + \text{Adv}_G^{\text{PRG}}(\mathcal{B}_2) + \text{Adv}_{W,1}^{\text{skPRF}}(\mathcal{B}_3) \\ &\quad + \text{Adv}_{\text{SE}}^{\text{OT-IND}\$}(\mathcal{B}_4), \end{aligned}$$

where S_1 is a simulator that returns a ciphertext $\tilde{c}_{\text{outOKEM}} \parallel \tilde{c}_{\text{inOKEM}}$ where $\tilde{c}_{\text{outOKEM}} \leftarrow \$ S_{\text{outOKEM}}$ and $\tilde{c}_{\text{inOKEM}} \leftarrow \$ \{0, 1\}^{\text{cl}_{\text{inOKEM}}}$, and

$$\text{Adv}_{\text{OEINC}}^{\text{SPR}\$-\text{CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{inOKEM}}^{\text{SPR}\$-\text{CCA}}(\mathcal{C}_1) + \text{Adv}_{W,2}^{\text{skPRF}}(\mathcal{C}_2) + \text{Adv}_{\text{outOKEM}}^{\text{strong-ctxt-unif}}(\mathcal{C}_3).$$

We note that when S_{outOKEM} is the simulator that samples ciphertexts uniformly at random from $\{0, 1\}^{\text{cl}_{\text{outOKEM}}}$, then the first case also satisfies SPR $\$$ -CCA.

Proof We separately consider the two advantage bounds via two proof cases.

Case I. In the first case, we reduce to SPR-CCA security of the outer OKEM, and proceed via a series of game hops.

Game I.0. We start with the security game for SPR-CCA ($\text{G}_{\text{OEINC}, S_1}^{\text{SPR-CCA}}(\mathcal{A})$).

Game I.1 In $\text{G}_{\text{I.1}}$, we replace (in the challenge encapsulation yielding K_1^* and c_1^*) the outer OKEM shared secret K_{out} and ciphertext c_{out} with a random key $\tilde{K}_{\text{out}} \leftarrow \$ \mathcal{K}_{\text{outOKEM}}$ and simulated ciphertext $\tilde{c}_{\text{out}} \leftarrow \$ S_{\text{outOKEM}}$. We bound the adversary's difference in advantage by a reduction $\mathcal{B}_1$ to the SPR-CCA of outOKEM w.r.t the simulator S_{outOKEM} . $\mathcal{B}_1$ obtains the SPR-CCA challenge (pk, c^*, K^*) for outOKEM and simulates the game as follows for $\mathcal{A}$. It uses pk in place of pk_{out} . When $\mathcal{O}_{\text{Decap}}$ is called, if $c_{\text{out}} = c^*$ (in line 17 of Fig. 4) then $\mathcal{B}_1$ computes the remainder of Decap using K^* as K_{out} ; else, if $c_{\text{out}} \neq c^*$ then $\mathcal{B}_1$ queries its SPR-CCA decapsulation oracle (for outOKEM) and uses the response as K_{out} .

If (c^*, K^*) are real values then $\mathcal{B}_1$ exactly simulates $\text{G}_{\text{I.0}}$ to $\mathcal{A}$; else, $\mathcal{B}_1$ simulates $\text{G}_{\text{I.1}}$ to $\mathcal{A}$. Therefore:

$$\Pr[\text{G}_{\text{I.0}}] - \Pr[\text{G}_{\text{I.1}}] \leq \text{Adv}_{\text{outOKEM}, S_{\text{outOKEM}}}^{\text{SPR-CCA}}(\mathcal{B}_1).$$

Game I.2 In $\text{G}_{\text{I.2}}$, for the computation of the challenge shared secret K_1^* , we replace the output $K_{\text{oe}}, K_{\text{ok}}$ of $\text{G}(\tilde{K}_{\text{out}})$ with uniformly random $\tilde{K}_{\text{oe}}, \tilde{K}_{\text{ok}}$, in particular for the challenge. We bound the difference in this step by a reduction $\mathcal{B}_2$ to the PRG security of G. The reduction uses its oracle in place of G, simulating either $\text{G}_{\text{I.1}}$ (if the G oracle output is real) or $\text{G}_{\text{I.2}}$ (if the G oracle output is random), giving:

$$\Pr[\text{G}_{\text{I.1}}] - \Pr[\text{G}_{\text{I.2}}] \leq \text{Adv}_G^{\text{PRG}}(\mathcal{B}_2).$$

<u>Encap(pk):</u> 1 $(pk_{out}, pk_{in}) \leftarrow pk$ 2 $(\tilde{K}_{out}, \tilde{c}_{out}) \leftarrow \\$K_{outOKEM} \times S_{outOKEM} \text{ // } G_{I.1}$ 3 $(\tilde{K}_{oe}, \tilde{K}_{ok}) \leftarrow \\$\{0, 1\}^{cl_{inOKEM}} \times \dots \text{ // } G_{I.2}$ 4 $(K_{in}, c_{in}) \leftarrow \\$inOKEM.Encap(pk_{in})$ 5 $c'_{in} \leftarrow SE.Enc(\tilde{K}_{oe}, c_{in})$ 6 $c \leftarrow c_{out} c'_{in}$ 7 $K \leftarrow \tilde{W}(K_{in}, c) \text{ // } G_{I.3}$ 8 return (K, c)	<u>Decap(sk, c):</u> 9 $(sk_{out}, sk_{in}) \leftarrow sk$ 10 $c_{out} c'_{in} \leftarrow c$ 11 if $c_{out} \neq \tilde{c}_{out}$ then ... // handle regularly // else: use $\tilde{K}_{oe}, \tilde{K}_{ok}$ 12 $c_{in} \leftarrow SE.Dec(\tilde{K}_{oe}, c'_{in})$ 13 $K_{in} \leftarrow inOKEM.Decap(sk_{in}, c_{in})$ 14 $K \leftarrow \tilde{W}(K_{in}, c) \text{ // } G_{I.4}: K_{in} \leftarrow 0$ 15 return K
--	--

Fig. 5. Modifications in the game hops $G_{I.1}$ – $G_{I.4}$ of the SPR-CCA proof, case (I), to the encapsulation and decapsulation algorithms of OEINC (Fig. 2, line 9 resp. 15).

Game I.3 We now replace evaluations of $W(\tilde{K}_{ok}, \cdot, \cdot)$ with a random function $\tilde{W}(\cdot, \cdot)$.⁴ This in particular replaces the challenge shared secret K with a uniformly random value, independent of outputs of the decapsulation oracle since the third input c to W must be distinct from the challenge c^* for each query.

We bound this step by a reduction $\mathcal{B}_3$ to the split-key pseudorandomness of W , where $\mathcal{B}_3$ uses its oracle in place of calls to $W(\tilde{K}_{ok}, \cdot, \cdot)$. It follows that

$$\Pr[G_{I.2}] - \Pr[G_{I.3}] \leq \text{Adv}_{W,1}^{\text{skPRF}}(\mathcal{B}_3).$$

Game I.4 Next, we rewrite the decapsulation oracle when queried with the challenge outer ciphertext $\tilde{c}_{out}$ in a way that is unnoticeable to the adversary. First, we fix the K_{in} input to $\tilde{W}$ (Fig. 5, line 14) to a zero string. Since K_{in} is deterministically derived from the second input $c = \tilde{c}_{out} || c'_{in}$ (as $\tilde{K}_{oe}$ and sk_{in} are fixed), this does not change the distribution of K : decapsulating a ciphertext $c = \tilde{c}_{out} || c'_{in}$ will yield distinct, randomly sampled shared secrets for distinct c'_{in} . Then $\Pr[G_{I.3}] = \Pr[G_{I.4}]$.

From this point on, we no longer need to decapsulate the corresponding inner ciphertext (lines 12 and 13 of Fig. 5 highlighted in gray), and hence also do not need to use $\tilde{K}_{oe}$ anymore to decrypt c'_{in} when answering decapsulation queries on ciphertexts containing the challenge outer ciphertext $\tilde{c}_{out}$.

Game I.5 Finally, in $G_{I.5}$ we replace the encrypted inner ciphertext c'_{in} of the challenge with random bits of length cl_{inOKEM} , by a reduction $\mathcal{B}_5$ to the OT-IND\$ security of SE. $\mathcal{B}_5$, on input c_{in} , obtains a challenge ciphertext c^* and simulates the game for $\mathcal{A}$, replacing c'_{in} with c^* in the challenge ciphertext. Importantly, $\mathcal{B}_5$ does not need to know the encryption key $\tilde{K}_{oe}$ to answer decapsulation queries on ciphertexts containing the challenge outer ciphertext $\tilde{c}_{out}$, as per $G_{I.4}$.

Hence $\mathcal{B}_5$ exactly simulates $G_{I.4}$ or $G_{I.5}$ to $\mathcal{A}$. Therefore:

⁴ The split-key PRF F_2 takes *three* distinguished inputs, two keys and one label (cf. Definition 1). Accordingly, the random function $\tilde{W}$ takes two distinguished inputs.

$$\Pr[G_{\text{I.4}}] - \Pr[G_{\text{I.5}}] \leq \text{Adv}_{\text{SE}}^{\text{OT-INDS}}(\mathcal{B}_5).$$

At this point, by $G_{\text{I.1}}$ and $G_{\text{I.5}}$, the ciphertext received by $\mathcal{A}$ is uniformly random and by $G_{\text{I.3}}$ the key is uniformly random. Therefore, $\text{Adv}_{\text{OEINC}, S_1}^{G_{\text{I.5}}}(\mathcal{A}) = 0$. Collecting the bounds yields the theorem statement.

Case II. In the second case, we reduce to SPR\$-CCA security of inOKEM and strong-ctxt-unif security of outOKEM.

Game II.0. We start with the security game for SPR\$-CCA ($G_{\text{OEINC}}^{\text{SPR\$-CCA}}(\mathcal{A})$).

Game II.1 In $G_{\text{II.1}}$, we replace (in the challenge encapsulation yielding K_1^* and c_1^*) the inner OKEM shared secret K_{in} and ciphertext c_{in} with a random shared secret $\tilde{K}_{in} \leftarrow \mathcal{K}_{\text{inOKEM}}$ and random ciphertext $\tilde{c}_{in} \leftarrow \{0, 1\}^{\text{cl}_{\text{inOKEM}}}$. The adversary $\mathcal{A}$'s advantage is then bounded by a reduction $\mathcal{C}_1$ to the SPR\$-CCA security of inOKEM, i.e., for a simulator that samples ciphertexts uniformly at random from $\{0, 1\}^{\text{cl}_{\text{inOKEM}}}$. Reduction $\mathcal{C}_1$ obtains an SPR\$-CCA challenge (pk, c^*, K^*) and simulates the game for $\mathcal{A}$ by using pk in place of pk_{in} . When $\mathcal{O}_{\text{Decap}}$ is called, if $c_{in} = c^*$ (in line 20 of Fig. 4), then $\mathcal{C}_1$ uses K^* in place of K_{in} ; else (when $c_{in} \neq c^*$), $\mathcal{C}_1$ queries its SPR\$-CCA decapsulation oracle (for inOKEM) and uses the response as K_{in} .

If (c^*, K^*) are real then $\mathcal{C}_1$ exactly simulates $G_{\text{II.0}}$ to $\mathcal{A}$. Otherwise, $\mathcal{C}_1$ simulates $G_{\text{II.1}}$ to $\mathcal{A}$. Thus,

$$\Pr[G_{\text{II.0}}] - \Pr[G_{\text{II.1}}] \leq \text{Adv}_{\text{inOKEM}}^{\text{SPR\$-CCA}}(\mathcal{C}_1).$$

Game II.2 Next, we replace evaluations of $W(\cdot, \tilde{K}_{in}, \cdot)$ with a random function. This in particular replaces K (both in the challenge computation and in the decapsulation oracle) with a uniformly random value, independent of outputs of the decapsulation oracle since the third input c to W is distinct from the challenge c^* for each query. This step is bounded by a reduction $\mathcal{C}_2$ to the split-key pseudorandomness of W , where $\mathcal{C}_2$ uses its oracle in place of calls to $W(\cdot, \tilde{K}_{in}, \cdot)$. Since $\tilde{K}_{in}$ is random by $G_{\text{II.1}}$, it follows that

$$\Pr[G_{\text{II.1}}] - \Pr[G_{\text{II.2}}] \leq \text{Adv}_{W, \tilde{2}}^{\text{skPRF}}(\mathcal{C}_2).$$

Game II.3 In $G_{\text{II.3}}$, we replace (in the challenge encapsulation) the encrypted inner ciphertext c'_{in} with a random ciphertext in $\{0, 1\}^{\text{cl}_{\text{inOKEM}}}$. This follows directly from the fact that $\text{SE.Enc}(K_{oe}, \cdot)$ is a permutation over $\{0, 1\}^{\text{cl}_{\text{inOKEM}}}$ and that $\tilde{c}_{in}$ is a random bitstring (by $G_{\text{II.1}}$). Therefore, $\Pr[G_{\text{II.2}}] = \Pr[G_{\text{II.3}}]$.

Game II.4 Now, the final shared secret K and the inner ciphertext are random (and independent of K_{out}), so it only remains to replace the outer ciphertext c_{out} with random. In $G_{\text{II.4}}$, we replace c_{out} with a randomly sampled ciphertext from $\{0, 1\}^{\text{cl}_{\text{outOKEM}}}$. We bound this change by a reduction $\mathcal{C}_3$ to the strong-ctxt-unif

of **outOKEM**. In particular, $\mathcal{C}_3$ simulates the game for $\mathcal{A}$ by using its challenge ciphertext c^* from the **strong-ctxt-unif** game in place of c_{out} . To answer $\mathcal{O}_{\text{Decap}}$ queries, $\mathcal{C}_4$ uses the given sk from the **strong-ctxt-unif** challenge to simulate calls to **outOKEM.Decap**(sk_{out}, c_{out}). Then, $\mathcal{C}_4$ exactly simulates $\mathbf{G}_{\text{II.3}}$ (if the **strong-ctxt-unif** challenge is real) or $\mathbf{G}_{\text{II.4}}$ (if the **strong-ctxt-unif** challenge is random) to $\mathcal{A}$. Therefore,

$$\Pr[\mathbf{G}_{\text{II.3}}] - \Pr[\mathbf{G}_{\text{II.4}}] \leq \text{Adv}_{\text{outOKEM}}^{\text{strong-ctxt-unif}}(\mathcal{C}_4).$$

Now, each of K , c_{out} , and c'_{in} are random. Hence, the adversary $\mathcal{A}$ can only guess the challenge bit b : $\text{Adv}_{\text{OEINC}}^{\mathbf{G}_{\text{II.4}}}(\mathcal{A}) = 0$. Collecting the bounds yields the theorem statement.

3.4 Ciphertext Uniformity

A hybrid ciphertext uniformity guarantee holds if the outer OKEM has statistical ciphertext uniformity. When the computational assumption underlying **inOKEM** is broken and the assumption underlying **outOKEM** holds, the outer KEM ciphertext is still uniformly random by its ciphertext uniformity, and the shared secret derived from **outOKEM** is used to encrypt the inner ciphertext, thus retaining its uniformity. This suggests relying on the **IND-CPA** security of the outer OKEM and its (regular) ciphertext uniformity. However, we observe that these two properties together are equivalent to **SPR\$-CPA** security and hence use the latter property, which simplifies the proof. Notably, the encryption step should not provide any additional information about the inner ciphertext; any use of authentication (such as AE or a MAC) would provide an oracle to the adversary distinguishing inner ciphertexts from random. On the other hand, if the **outOKEM** assumption is broken, then the encryption key no longer provides any guarantee, so ciphertext uniformity relies on that of both the outer and inner ciphertexts.

Theorem 3 (reg-ctxt-unif of OEINC). *Let $\text{OEINC} = \text{OEINC}[\text{outOKEM}, \text{inOKEM}, \text{SE}, \text{G}, \text{W}]$ be a combined OKEM as defined in Fig. 4, where SE is a length-preserving symmetric encryption scheme, G a PRG, and W a split-key PRF. For any adversary $\mathcal{A}$ against the regular ciphertext uniformity (Definition 7) of OEINC, we give algorithms $\mathcal{B}_1$ – $\mathcal{B}_3$ and $\mathcal{C}_1$ – $\mathcal{C}_2$ such that*

$$\text{Adv}_{\text{OEINC}}^{\text{reg-ctxt-unif}}(\mathcal{A}) \leq \text{Adv}_{\text{outOKEM}}^{\text{SPR\$-CPA}}(\mathcal{B}_1) + \text{Adv}_{\text{G}}^{\text{PRG}}(\mathcal{B}_2) + \text{Adv}_{\text{SE}}^{\text{OT-IND\$}}(\mathcal{B}_3)$$

and

$$\text{Adv}_{\text{OEINC}}^{\text{reg-ctxt-unif}}(\mathcal{A}) \leq \text{Adv}_{\text{inOKEM}}^{\text{reg-ctxt-unif}}(\mathcal{C}_1) + \text{Adv}_{\text{outOKEM}}^{\text{reg-ctxt-unif}}(\mathcal{C}_2).$$

3.5 Strong Second Pseudorandomness

We show that our combiner provides strong second pseudorandomness so that we may use it in the CAKE password-authenticated key exchange protocol (Sect. 5).

At a high level, this property operates similarly to ciphertext uniformity. A hybrid strong second pseudorandomness guarantee holds if the outer OKEM has statistical strong second pseudorandomness. If the assumption underlying inOKEM is broken and that underlying outOKEM is not, then the outer key is sufficient to obfuscate the inner ciphertext. And if outOKEM's IND-CPA property is broken, then the resulting strong-2nd-pr property relies on the strong-2nd-pr of both outOKEM and inOKEM.

Theorem 4 (Strong second pseudorandomness of OEINC). *Let $\text{OEINC} = \text{OEINC}[\text{outOKEM}, \text{inOKEM}, \text{SE}, \text{G}, \text{W}]$ be a combined OKEM as defined in Fig. 4. For any adversary $\mathcal{A}$ against the strong second pseudorandomness (Definition 8) of OEINC, we give algorithms $\mathcal{B}_1$ – $\mathcal{B}_4$ and $\mathcal{C}_1$ – $\mathcal{C}_2$ such that*

$$\begin{aligned} \text{Adv}_{\text{OEINC}}^{\text{strong-2nd-pr}}(\mathcal{A}) &\leq \delta_{\text{outOKEM}} + 2 \cdot \left(\text{Adv}_{\text{outOKEM}}^{\text{IND-CPA}}(\mathcal{B}_1) + \text{Adv}_{\text{G}}^{\text{PRG}}(\mathcal{B}_2) \right) \\ &\quad + \text{Adv}_{\text{SE}}^{\text{OT-IND\$}}(\mathcal{B}_3) + \text{Adv}_{\text{outOKEM}}^{\text{strong-2nd-pr}}(\mathcal{B}_4), \end{aligned}$$

and

$$\text{Adv}_{\text{OEINC}}^{\text{strong-2nd-pr}}(\mathcal{A}) \leq \delta_{\text{outOKEM}} + \delta_{\text{inOKEM}} + \text{Adv}_{\text{inOKEM}}^{\text{strong-2nd-pr}}(\mathcal{C}_1) + \text{Adv}_{\text{outOKEM}}^{\text{strong-2nd-pr}}(\mathcal{C}_2).$$

3.6 Public Key Uniformity

Finally, OEINC only has public key uniformity as long as *both* underlying OKEMs retain their public key uniformity. Since no shared key material is available when distributing a public key, there is little that would help in obfuscating a public key if the underlying assumption is broken. Thus, if one of the schemes does not have statistical public key uniformity, there does not appear to be a way to construct a scheme with hybrid public key uniformity, barring some additional PKI-like infrastructure. OEINC does achieve public key uniformity conditioned on the public key uniformity of *both* underlying schemes:

Theorem 5 (pk-unif of OEINC). *Let $\text{OEINC} = \text{OEINC}[\text{outOKEM}, \text{inOKEM}, \text{SE}, \text{G}, \text{W}]$ be a combined OKEM as defined in Fig. 4. For any adversary $\mathcal{A}$ against the public key uniformity (Definition 6) of OEINC, we give algorithms $\mathcal{B}_1$, $\mathcal{B}_2$ such that*

$$\text{Adv}_{\text{OEINC}}^{\text{pk-unif}}(\mathcal{A}) \leq \text{Adv}_{\text{outOKEM}}^{\text{pk-unif}}(\mathcal{B}_1) + \text{Adv}_{\text{inOKEM}}^{\text{pk-unif}}(\mathcal{B}_2).$$

3.7 Instantiating the Combiner

Following our results for OEINC, we now know that we can construct an OKEM with hybrid guarantees when the outer OKEM has statistical ciphertext uniformity. Previous work [34] constructed an MLWE-based OKEM, ML-Kameleon, from ML-KEM and Kameleon encodings, notably whose ciphertext (and public key) uniformity depends on the MLWE assumption, i.e., it is not statistical. Similarly, Saber [25] and FrodoKEM [21] achieve uniformity from MLWR and

LWE assumptions, respectively. These are natural choices for the inner OKEM. As for the outer OKEM, we can construct a (classically-secure) OKEM using DHKEM [13] and Elligator2 [18] or Elligator² [59] encodings. We provide the details of this construction, denoted DHKEM-Ell2, in the full version [33]. Importantly, Elligator-style encodings do not rely on any underlying computational assumptions for uniformity guarantees, thus giving the resulting OKEM *statistical* ciphertext uniformity. Both ML-Kameleon and DHKEM-Ell2 are SPR\$-CCA-secure. The symmetric encryption SE can be instantiated with XOR. Therefore, instantiating OEINC[DHKEM-Ell2, ML-Kameleon, $\oplus$, G, W] for any reasonable choice of PRG G and split-key PRF W (e.g., see [20, 31]) provides the desired hybrid guarantees.

Table 1. Security properties of combiners instantiated with different input (O)KEMs. Underlying computational assumptions are given in parentheses. Dots in the table indicate: (at least) one of the underlying assumptions must hold, i.e., hybrid guarantee (●); the assumption of the outer KEM must hold (◐); both underlying assumptions must hold (⊙); or the property does not hold (–). For SPR-CCA, subscripts indicate whether the respective simulator samples bitstrings (\$) or uniform ciphertexts from the ciphertext space (C).

Combined KEM		IND-CCA	SPR-CCA	reg-ctxt-unif	pk-unif
OEINC Outer and Inner KEM					
DHKEM-Ell2(GapDH)	ML-Kameleon(MLWE)				
	Saber (MLWR)	●	● _{\$}	●	⊙
	FrodoKEM (LWE)				
ML-Kameleon(MLWE)	DHKEM-Ell2(GapDH)	●	◐ _{\$}	◐	⊙
DHKEM-Ell2(GapDH)	ML-KEM(MLWE)	●	◐ _{\$}	◐	–
DHKEM(GapDH)	ML-KEM(MLWE)	●	◐ _C	–	–
X-Wing [12] (MLWE, GapDH)		●	⊙ _C	–	–
Parallel Combiner [20, 31] ML-Kameleon + DHKEM-Ell2 (MLWE, GapDH)		●	⊙ _{\$}	⊙	⊙

We now explore alternative options for instantiating the combined OKEM, omitting any further discussion of SE, G, and W, as they can be instantiated with $\oplus$, any secure PRG, and any split-key PRF, respectively. Table 1 summarizes the properties of each of the following variants, and compares them with the security properties provided by X-Wing [12] (which does not use obfuscated KEMs) and the generic parallel combiner [20, 31] instantiated with two obfuscated KEMs.

Non-obfuscated KEMs. The OEINC construction is not restricted to only OKEMs. Ignoring the step that encodes public keys, one can use any KEM as an input to the combiner. It is then interesting to ask which properties are maintained by the combiner in such cases. Of course, using any two IND-CCA secure KEMs (e.g., ML-KEM and DHKEM) results in an IND-CCA combined KEM. The remaining properties depend on the input KEMs. For example, using ML-KEM as the inner KEM and DHKEM as the outer KEM, the combined KEM would not achieve public key or ciphertext uniformity; however, it does achieve SPR-CCA security with respect to a simulator that outputs structured ciphertexts (i.e., DH values concatenated with random bitstrings resulting from encrypting ML-KEM ciphertexts), assuming that the outer KEM retains its SPR-CCA security. If the assumption underlying DHKEM is broken, then this construction no longer achieves SPR-CCA security, regardless of the security of the inner KEM.

Outer OKEM and inner KEM. Alternatively, one might ask what happens if only the outer KEM is obfuscated and the inner KEM is not. To illustrate, we consider the combination of DHKEM-ElI2 as the outer KEM and ML-KEM as the inner KEM. It is easy to see that IND-CCA security is maintained as a hybrid guarantee and that public key uniformity is no longer achieved. SPR-CCA security is achieved if the outer OKEM is SPR-CCA-secure, but the combination does not provide a hybrid guarantee. There is no longer a hybrid guarantee for ciphertext uniformity, which now relies on the IND-CPA security and ciphertext uniformity of DHKEM-ElI2 (since ML-KEM has no ciphertext uniformity).

Outer OKEM without strong-ctxt-unif. Finally, we ask what happens if we use ML-Kameleon as the outer KEM and DHKEM-ElI2 as the inner KEM (swapping the order of our initial proposed instantiation). The (hybrid) IND-CCA and (non-hybrid) public key uniformity guarantees remain the same as in the original case. SPR-CCA is not a hybrid guarantee anymore and now depends on the SPR-CCA security of ML-Kameleon, since ML-Kameleon does not have statistical ciphertext uniformity. In particular, when the outer assumption breaks down, the key used for encryption may no longer be random, so even though the inner ciphertexts are simulatable, it is unclear how to simulate the (not necessarily random) keyed permutation of (non-uniform) simulatable ciphertexts. Similarly, ciphertext uniformity requires now that the ciphertext uniformity (and IND-CPA security) of ML-Kameleon is maintained (i.e., no hybrid guarantee).

4 A Hybrid Obfuscated Key Exchange Protocol

We now present `Drive1`, a revised version of the post-quantum variant `pq-obfs` [34] of Tor's `obfs4` protocol that enables hybrid security guarantees. We recall the purpose of an obfuscated key exchange protocol [34] is to permit a client with prior knowledge of a server's public key to establish a fresh session key with the server, such that (1) no passive adversary can distinguish the handshake from a random sequence of messages of the same length and ordering; and (2) the server is otherwise quiet, meaning it does not respond to any messages sent by

an adversary that does not demonstrate knowledge of the server’s public key. The intention of the **obfs4** protocol in particular is to provide a layer of obfuscated traffic that does not match a censor’s protocol blacklist and is secure against probing by censors. We state the security goals in more detail below.

1. **Key indistinguishability.** A Bellare–Rogaway-style [16] notion of key indistinguishability requires that the shared key derived in the protocol must be indistinguishable from random. Formally, an adversary actively interacts with multiple sessions, is allowed to reveal user and session keys, and must guess the challenge bit b used in a **TEST** oracle which returns either real or random session keys depending on b . (A **Fresh** predicate prevents trivial attacks like testing and revealing the same key.) We also aim for *forward secrecy*: session keys must remain secure if the involved server’s long-term secret key is later compromised.
2. **Obfuscation.** The core goal of obfuscated key exchange is to offer protection against passive deep packet inspection by hiding protocol fingerprints. The protocol transcript should therefore be indistinguishable from a simulated transcript, where the *simulator* $\mathcal{S}$ is a parameter of the security definition. Using a simulation-based definition allows one to capture additional structure of the key exchange transcript (e.g., packet lengths, entropy, etc.). In particular, protocols can be divided into *classes* corresponding to simulators, where a class determines its traffic pattern. As with **pq-obfs**, we prove obfuscation with respect to a simulator that outputs random messages of varying lengths. This captures the idea that the protocol should lie within the class of protocols where all transmitted data appears to be random bits. An adversary’s goal is to guess the challenge bit b used in a **CHALLENGE** oracle which returns either real or simulated transcripts of the protocol (and real or random keys, respectively). The challenge bit b is the same as is used in **TEST** queries. The adversary must not violate the **ObfFresh** predicate, which requires that the server’s secret key is not revealed (i.e., we aim for *strong obfuscation* as defined in [34]).
3. **Probing resistance.** The protocol should be resistant to *active probing*, where censors probe suspicious proxy servers in an attempt to identify them. A responder should not respond to messages from an initiator who has not proven knowledge of the responder’s public key. This is captured by the **SEND** oracle and the **Probed** flag: If the adversary successfully elicits a non-empty response from a responder, whose public key is not revealed, in response to a message not previously sent by an honest initiator, then the adversary wins.
4. **Explicit authentication.** Finally, **ExplicitAuth** ensures explicit authentication of the server. An adversary wins if it causes a client to accept a session without a partnered server session existing.

4.1 Shortcomings of the **pq-obfs** Protocol

We identify a key issue that prevents **pq-obfs** from achieving hybrid guarantees, even when instantiated with a hybrid OKEM. In the first round of **pq-obfs**, the

client, who knows the server’s static KEM public key pk_S , encapsulates to pk_S , yielding (c_S, K_S) and generates an ephemeral keypair $(sk_e, pk_e) \leftarrow \text{KEM.KGen}()$. In the first message, the client sends c_S and pk_e to the server. The server will decapsulate c_S and use pk_e to establish a fresh session key with the client. The issue with this approach is that pk_e is sent to the server *in the clear*, requiring public key uniformity from the ephemeral OKEM. However, we do not know of an existing OKEM with *hybrid* public key uniformity guarantees. In other words, **pq-obfs** with a hybrid OKEM has obfuscation that is only as strong as the weakest pk-unif of its underlying OKEMs.

4.2 The Drive1 Protocol

We propose the **Drive1** protocol which addresses the above hybrid obfuscation barrier of **pq-obfs** while achieving the same security goals. The key insight is to encrypt the ephemeral public key and ciphertext using a key derived from the shared secret from the static OKEM encapsulation. That way, **Drive1**, unlike **pq-obfs**, does not require public-key uniformity of the deployed (hybrid) ephemeral KEM, which is the only OKEM property for which OEINC does not achieve hybrid guarantees. Furthermore, additionally encrypting the ephemeral ciphertext means that for the ephemeral KEM, a regular (non-obfuscated) KEM suffices, improving efficiency. Instantiating **Drive1** with $\text{OEINC}[\text{DHKEM-EI2}, \text{ML-Kameleon}, \oplus, \mathbf{G}, \mathbf{W}]$ (for reasonable PRG $\mathbf{G}$ and PRF $\mathbf{W}$) as the static KEM and any hybrid IND-1CCA-secure (non-obfuscated) KEM as the ephemeral KEM hence yields a readily implementable hybrid obfuscated key exchange protocol. The final structure remains closely aligned with **pq-obfs** and the currently deployed **obfs4** protocol.

We retain from **pq-obfs** the general structure of ephemeral and static KEM encapsulations and the key schedule (modulo deriving the additional encryption keys), so that we need only rely on standard assumptions (avoiding any random oracles). The full description can be found in Fig. 6. We assume that the server has published its OKEM public key and an identifier $NodeID \in \{0, 1\}^n$, to a distribution service that only honest clients have access to. In practice, this can be a *bridge distribution service*, as used with Tor, or another out-of-band sharing mechanism. Although pre-shared semi-private information is a seemingly strong assumption, it is already an assumption of **obfs4**.

Client to Server Message. A client who knows the server’s public key and $NodeID$ begins by generating a fresh ephemeral KEM keypair. It then establishes a shared secret with the server by encapsulating to the server’s static OKEM public key. The client sends this ciphertext along with the ephemeral public key (encrypted, using a key derived from the static OKEM encapsulation), randomized padding, and MACs which include the server’s public key and $NodeID$.

Server to Client Message. The server decapsulates and checks the MACs. On success, it encapsulates to the client’s ephemeral public key, and sends back the ciphertext (encrypted using a key derived from the static OKEM encapsulation), along with padding and MACs. The client verifies these and derives the session

key using the ephemeral-ephemeral and ephemeral-static KEM shared secrets, as well as the corresponding public keys and ciphertexts.

MACs. The MAC tags M_C and M_S are intended to aid in parsing the variable-length messages. In particular, the server may continue sending data immediately following its message to the client, and so the M_S marker identifies where the initial handshake message (and variable length padding) ends. The client's tag M_C is not strictly necessary, as the server may parse the handshake message from the end; however, we leave it in the description for symmetry and to allow for potentially more flexible designs in the future.

Replay Protection. We assume that, for replay protection, the protocol works in epochs; the client includes its epoch in MAC tag MAC_C . The server checks that this epoch is within a valid range, and stores the set of MAC values seen in its state $\mathcal{S}_{MAC}$ per epoch. To simplify the presentation, we omit epochs here.

Instantiating the Protocol. `Drive1` relies on a secure pseudorandom function F_1 with (default) output length fl_1 and a dual-PRF F_2 with output length fl_2 . In addition, we employ an OT-IND\$-secure symmetric encryption scheme SE . To handle potentially varying key lengths, we ask that F_1 also supports variable output lengths, where the output length is optionally specified through the function's third input (by default, it is fl_1). Any IND-1CCA-secure KEM can be used for the ephemeral KEM, and any OKEM with IND-CCA security, SPR\$-CCA security, and ciphertext uniformity can be used for the static OKEM.

This can be achieved, for example, with an obfuscated KEM constructed using OEINC (see Table 1 for examples). The ephemeral KEM can be instantiated with any regular (i.e., non-obfuscated) KEM with IND-1CCA security (e.g., X-Wing [12] would provide hybrid guarantees). We can instantiate F_1 with HKDF-Expand [44, 45] and F_2 with HMAC [15, 43]; the latter's dual-PRF security for fixed-length keys is proven in [9]. The symmetric encryption scheme, requiring only OT-IND\$ security, can be instantiated with XOR with appropriate key lengths kl_1, kl_2 matching the ephemeral KEM's public key and ciphertext lengths.

4.3 Security

We use the security model of obfuscated key exchange from [34], denoted `sObfKE`. In addition to the details of the model specific to the desired security properties described above, session identifiers are used to determine that two sessions are partnered, while contributive identifiers are used to determine when a responder session has an honest communication partner. The security analysis makes use of a selective security variant of the game, where the adversary has to commit upfront to winning via (1) a single TEST query to a pre-declared session, or (2) a single CHALLENGE query against a pre-declared server. This variant implies full security via a hybrid argument, as shown in [34].

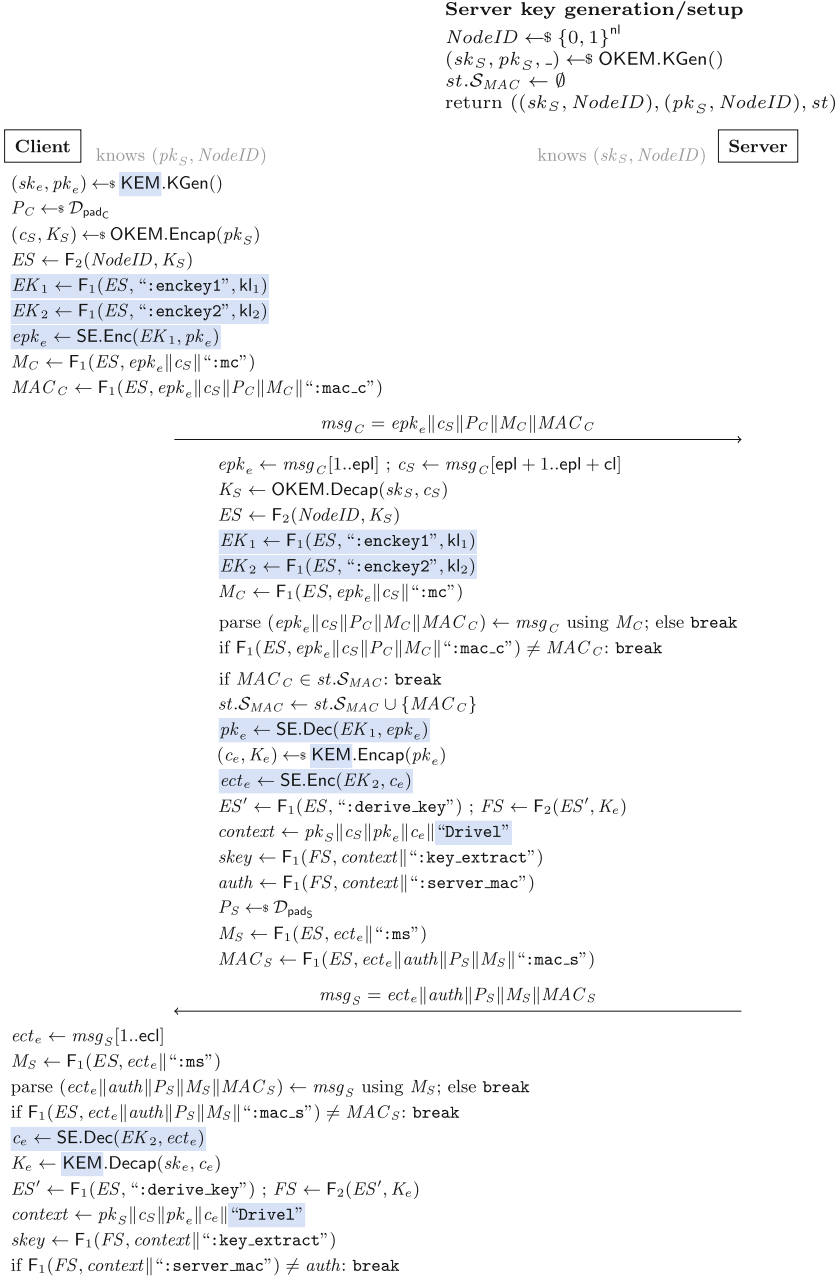


Fig. 6. The *Drivel* obfuscated key exchange protocol. OKEM is an OKEM satisfying IND-CCA, SPR-CCA, and ciphertext uniformity. KEM is an IND-1CCA-secure KEM. SE is an OT-IND-secure symmetric encryption scheme. F_1 is a PRF and F_2 is a dual PRF. Core differences to the *pq-obfs* protocol from [34] are highlighted in blue boxes. Values *epl*, *ecl*, *cl* are the lengths of the encrypted ephemeral KEM public key and ciphertext as well as OKEM ciphertext, respectively; kl_1 , kl_2 the encryption key lengths.

Simulator Definition. We establish sObfKE security with respect to the following simulator $\mathcal{S}_{\text{Drivel}}$ which outputs two uniformly random messages of length dependent on the Drivel distribution of padding:

$\mathcal{S}_{\text{Drivel}}:$

```

1  $P_C \leftarrow \$ \mathcal{D}_{\text{pad}_C}$  ;  $P_S \leftarrow \$ \mathcal{D}_{\text{pad}_S}$  // sample client/server padding according to distribution
2  $m_1 \leftarrow \$ \{0, 1\}^{\text{epl}+\text{cl}+2\cdot\text{fl}_1+|P_C|}$  // 1x encrypted pk + 1x ctxt + 2x  $F_1$  outputs (MACs) + client padding
3  $m_2 \leftarrow \$ \{0, 1\}^{\text{ecl}+3\cdot\text{fl}_1+|P_S|}$  // 1x encrypted ctxt + 3x  $F_1$  outputs (auth and MACs) + server padding
4 return  $(m_1, m_2)$ 
```

Here, epl, ecl are the lengths of the encrypted ephemeral KEM public key and ciphertext, respectively, fl_1 is the (default) output length of the PRF F_1 , and cl is the ciphertext length of OKEM.

Session and Contributive Identifiers. We set the session identifier as $\text{sid} := (pk_e, c_e, pk_S, c_S)$ where pk_e is the initiator's KEM public key and c_e is the corresponding KEM ciphertext, and pk_S is the responder's static KEM public key and c_S the corresponding ciphertext. We set the contributive identifier to $\text{cid} := (pk_e, c_S)$ upon the client sending or server receiving the first message.

Theorem 6. *Let Drivel be defined as in Fig. 6. For any sObfKE adversary $\mathcal{A}$ against Drivel, we give algorithms $\mathcal{B}_1\text{--}\mathcal{B}_{17}$ such that*

$$\begin{aligned}
\text{Adv}_{\text{Drivel}, \mathcal{S}_{\text{Drivel}}}^{\text{sObfKE}}(\mathcal{A}) \leq & 2 \cdot \left(\text{pkcoll}_{\text{KEM}}(n_s) + \text{pkcoll}_{\text{OKEM}}(n_r) + n_s \cdot (\delta_{\text{OKEM}} + \delta_{\text{KEM}}) \right. \\
& + n_s n_r \cdot \left(\text{Adv}_{F_2}^{\text{PRF}}(\mathcal{B}_1) + \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_2) + \frac{1}{2^{\text{fl}_1}} \right) \\
& + n_s n_r \cdot \left(\text{Adv}_{\text{OKEM}}^{\text{IND-CCA}}(\mathcal{B}_3) + \text{Adv}_{F_2}^{\text{swap-PRF}}(\mathcal{B}_4) + \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_5) \right. \\
& \quad \left. + \text{Adv}_{F_2}^{\text{PRF}}(\mathcal{B}_6) + \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_7) + \frac{1}{2^{\text{fl}_1}} \right) \\
& + (n_s + n_r q_C) \cdot \left(n_s \cdot \left(\text{Adv}_{\text{KEM}}^{\text{IND-1CCA}}(\mathcal{B}_8) + \text{Adv}_{F_2}^{\text{swap-PRF}}(\mathcal{B}_9) + \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_{10}) \right) \right. \\
& \quad + \text{Adv}_{\text{OKEM}}^{\text{SPRS-CCA}}(\mathcal{B}_{11}) + \text{Adv}_{F_2}^{\text{swap-PRF}}(\mathcal{B}_{12}) \\
& \quad + \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_{13}) + \text{Adv}_{F_2}^{\text{PRF}}(\mathcal{B}_{14}) + \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_{15}) \\
& \quad \left. \left. + \text{Adv}_{\text{SE}}^{\text{OT-IND\$}}(\mathcal{B}_{16}) + \text{Adv}_{\text{SE}}^{\text{OT-IND\$}}(\mathcal{B}_{17}) \right) \right),
\end{aligned}$$

where $\mathcal{A}$ makes at most q_C CHALLENGE queries; n_s, n_r are the number of sessions and servers (parties in responder role) that $\mathcal{A}$ interacts with, respectively; fl_1 is the output bit-length of F_1 ; and δ_{OKEM} and δ_{KEM} are the correctness errors of the obfuscated and ephemeral KEM, respectively.

Due to the intentional similarity of Drivel to pq-obfs, the proof of Theorem 6 follows closely (in large parts verbatim) the security proof for pq-obfs from [34]. Here, we provide a proof sketch with the main changes compared to the proof for pq-obfs underlined; the complete proof is deferred to the full version [33].

Proof Sketch. We proceed via a series of game hops and branches, considering each clause in FINALIZE. We rule out KEM public key collisions (term $\text{pkcoll}_{\text{KEM}}(n_s) + \text{pkcoll}_{\text{OKEM}}(n_r)$) and assume correct decapsulation in all sessions ($n_s \cdot (\delta_{\text{OKEM}} + \delta_{\text{KEM}})$), which ensures soundness (Sound).

A first branch then rules out that $\mathcal{A}$ successfully probes a server (Probed): we guess the first server and session that sets Probed (with a $n_s \cdot n_r$ loss). From that server's uncompromised semi-private *NodeID*, we apply PRF security twice to turn *ES* and then MAC_C into random values. After this, we observe that Probed is set if a non-initiator-first message m yields a reply by an unrevealed responder. Such a message m is either rejected due to the replay check, or it is different from all honest initiators' first messages sent to this responder. In the second case, m contains the target client MAC value MAC_C^* which is a random fl_1 -bit value unknown to $\mathcal{A}$. The adversary $\mathcal{A}$ can therefore only guess MAC_C^* ; the corresponding bound is $\text{Adv}_{\text{F}_2}^{\text{PRF}}(\mathcal{B}_1) + \text{Adv}_{\text{F}_1}^{\text{PRF}}(\mathcal{B}_2) + 1/2^{n_1}$.

A second branch prevents explicit authentication (ExplicitAuth) from being violated. We guess the first session violating ExplicitAuth, π^* , and its peer, v^* , (losing a factor $n_s \cdot n_r$). Then we can embed an IND-CCA challenge in that the peer's public key, the session's c_S , and the derived K_S shared secret (term $\text{Adv}_{\text{OKEM}}^{\text{IND-CCA}}(\mathcal{B}_3)$). Let $(c_S, K_S) \leftarrow \text{OKEM.Encap}(pk_S)$ be the encapsulation computed in the target session π^* with the long-term public key of v^* . We replace the long-term shared secret K_S with a uniformly random $\tilde{K}_S$ in π^* . All values derived from K_S in π^* use the randomized value $\tilde{K}_S$. We bound the adversary $\mathcal{A}$'s difference in advantage by a reduction $\mathcal{B}_3$ to the IND-CCA security of OKEM. Next, through four (swap-)PRF hops ($\mathcal{B}_4$ – $\mathcal{B}_7$), we show that this turns *auth* into a random fl_1 -bit value leaving $\mathcal{A}$ with a $1/2^{n_1}$ chance to violate ExplicitAuth.

We then restrict the adversary to a single-challenge (sObfKE-1) version of the game, applying the hybrid argument from [34, Theorem 4.3] with a $(n_s + n_r q_C)$ loss. We separately treat the following two cases.

For Case I ($\mathcal{A}$ makes a single TEST query), we first guess the (*sid*- or *cid*-) partner session of the tested session (losing a factor n_s). We then embed an IND-1CCA challenge in the ephemeral KEM encapsulation of the test session, π^* , and its partner session, π_p^* (term $\text{Adv}_{\text{KEM}}^{\text{IND-1CCA}}(\mathcal{B}_8)$). In particular, we replace the ephemeral KEM key K_e with a uniformly random $\tilde{K}_e$ in the target session π^* . All values derived from K_e in π^* use the randomized value $\tilde{K}_e$. We bound the adversary $\mathcal{A}$'s difference in advantage by a reduction $\mathcal{B}_8$ to the IND-1CCA security of OKEM. Following this, we apply two (swap-)PRF hops ($\mathcal{B}_9, \mathcal{B}_{10}$) to randomize the test session's key, concluding this case.

For Case II ($\mathcal{A}$ makes a single CHALLEXEC query to a pre-determined server), we embed an SPR\$-CCA challenge in the long-term KEM encapsulation of the CHALLEXEC sessions (term $\text{Adv}_{\text{OKEM}}^{\text{SPR\$-CCA}}(\mathcal{B}_{11})$), randomizing both c_S and K_S . Specifically, we replace the ciphertext c_S , as well as the long-term KEM key K_S as follows. Instead of running OKEM.Encap , the initiator samples $c_S \leftarrow \{0, 1\}^{\text{cl}}$ and $K_S \leftarrow \mathcal{K}$ uniformly at random. We bound the adversary $\mathcal{A}$'s difference in advantage by a reduction $\mathcal{B}_{11}$ to the SPR\$-CCA security of OKEM. Subsequently, we proceed via four (swap-)PRF hops ($\mathcal{B}_{12}$ – $\mathcal{B}_{15}$) to randomize

the transcript's MAC values as well as the encryption keys and session key. The main difference from the proof of **pq-obfs** is that in the final steps, we apply the $\text{OT-IND}^\$$ security of the symmetric encryption scheme **SE** to randomize epk_e and ect_e , corresponding to the terms $\text{Adv}_{\text{SE}}^{\text{OT-IND}^\$}(\mathcal{B}_{16})$ and $\text{Adv}_{\text{SE}}^{\text{OT-IND}^\$}(\mathcal{B}_{17})$. At this point, the protocol messages in **CHALLENGE** sessions are uniformly distributed like outputs of the simulator $\mathcal{S}_{\text{Drive1}}$, concluding this case and the proof.

Hybrid Security. **Drive1** achieves hybrid (classic / quantum-safe) security when using a hybrid construction for the involved OKEM and KEM. (Note that all other components of **Drive1** are symmetric.) Instantiating the OKEM with **OEINC**, advantage terms corresponding to OKEM can be unfolded using the bounds established in Sect. 3, yielding corresponding hybrid bounds for **Drive1**.

4.4 Additional Features

We now discuss optional features of **Drive1** which are intended to aid in development and deployment, but are not included in our security analysis.

Strong Obfuscation vs. DoS Resistance. **Drive1**, as described thus far, has *strong obfuscation*, meaning that even an adversary who learns the server's static public key pk_s is not able to passively distinguish a protocol's transcript from a simulated one. The cost of this advantage is that the server must perform a decapsulation for every incoming message, even when the other party does not know the server's *NodeID*. Since decapsulations can be costly, this may be used by an attacker to launch a denial-of-service attack against the server.⁵

In order to accommodate different deployment environments, the client can optionally send a *cookie* in the first round of communication, as was done in the original **obfs4** protocol as well as Wireguard [26]. Concretely, this cookie may be computed as $\text{HMAC}(pk_s \| \text{NodeID}, epk_e \| c_S \| P_C \| M_C \| MAC_C)$ (the difference from MAC_C being that a static key $pk_s \| \text{NodeID}$ is used rather than an ephemeral key ES). The server can then check this cookie prior to performing decapsulation, preventing the processing of messages from clients that do not know *NodeID*. To prevent replays, the server must store past cookies.

Padding Distribution. We enable an arbitrary distribution of padding to be used. An arbitrary padding distribution provides more flexibility in traffic patterns, allowing, for example, a padding distribution that fixes message lengths to match the traffic pattern of another protocol.

Extra Data. One may optionally place data encrypted under a key derived from K_S in place of (or in addition to) the padding of the initial message to the server. This is analogous to the *extra data* feature of **ntor** version 3 [50]. We note that this extra data does not benefit from forward secrecy because it is encrypted under a key derived from the static KEM encapsulation alone.

⁵ This tradeoff between strong obfuscation and DoS resistance in **obfs4** has been previously noted [6].

5 Hybrid PAKE with Adaptive Security

In this section we describe a method for constructing a password-authenticated key exchange (PAKE) from an OKEM. When the OKEM has hybrid guarantees, then so does the resulting PAKE. We distinguish our approach from previous constructions [39, 48] in two ways. First, our approach achieves adaptive, rather than static, security, since the underlying PAKE protocol is proven to realize the $\mathcal{F}_{\text{PAKE}}$ ideal functionality in the UC model [22] with adaptive corruptions. Second, our approach achieves a new tradeoff between round complexity and runtime: previous hybrid PAKE constructions include efficient three-round protocols which use standard lattice-based KEMs, as well as more computationally intensive one-round protocols using isogenies; our protocol has two rounds, and has computational overhead that lies in between the two.

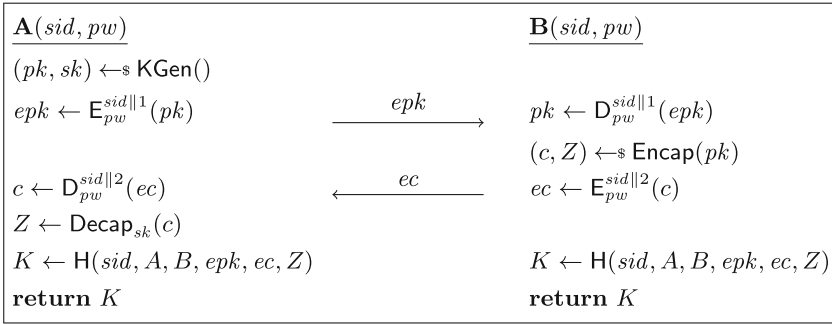


Fig. 7. The CAKE PAKE [14]. (E, D) are the encryption and decryption algorithms of an ideal cipher. sid is the protocol execution’s session identifier, and A, B are the party identifiers.

5.1 CAKE

We first provide an overview of CAKE [14], an adaptively-secure PAKE that is generically constructed from an underlying KEM with certain properties. We observe that an OKEM with ciphertext and public key uniformity is sufficient to instantiate CAKE. Then, we construct a hybrid OKEM with unconditional public key uniformity. Together, this yields the first hybrid adaptively-secure PAKE.

Protocol. An overview of the CAKE protocol is given in Fig. 7. The initiator generates a fresh KEM keypair, encrypting the public key using an ideal cipher with pw as the key, and sends the result to the responder. The responder decrypts the public key, encapsulates to it, and encrypts the encapsulation using the ideal cipher with the same password. Finally, the initiator decrypts and decapsulates. The session key is the hash of the KEM shared secret and the protocol transcript.

Required KEM Properties. CAKE requires its underlying KEM to have IND-CPA, strong pseudorandomness ([42, Definitions 2.3–2.5]), anonymity [14,

Definition 4], and fuzziness [14, Definition 3]. When public keys and ciphertexts are bitstrings, anonymity is strictly weaker than ciphertext uniformity and public key uniformity (Definition 6) is equivalent to fuzziness (these claims are treated more rigorously in the full version [33]). Finally, [42] defines strong pseudorandomness as ciphertext uniformity plus strong second pseudorandomness.

Thus, CAKE can be instantiated with an OKEM with IND-CPA security, ciphertext uniformity, strong second pseudorandomness, and public key uniformity. Further, since encoded public keys are uniform-looking bitstrings, we may instantiate the ideal cipher with a tweakable wide block cipher, and use the tweak for domain separation. This is easier to implement and more efficient than, for example, a custom, constant-time, keyed permutation on the range $[0, q^{768})$, as suggested by the CAKE authors for ML-KEM-768 public keys.

5.2 Achieving Hybrid CAKE

Using the OKEM combiner of Sect. 3, applied to, e.g., DHKEM-ElI2 (outer) and ML-Kameleon (inner), we obtain an OKEM with hybrid IND-CCA (and, hence, IND-CPA) security and ciphertext uniformity, but not public key uniformity (and possibly also not strong second pseudorandomness). Recall that the public key uniformity of the combined OKEM is the *weakest* of the underlying public key uniformity properties. Thus, to construct CAKE with hybrid guarantees, we will need to combine two OKEMs with statistical public key uniformity. Since our outer KEM already has this property, it remains only to construct a post-quantum OKEM with statistical public key uniformity. No post-quantum OKEM we have mentioned so far achieves this; however, we are able to construct one based on unstructured LWE. We follow the technique hinted at in [39] for constructing a one-round lattice-based PAKE from (superpolynomial modulus) LWE.

Post-quantum Statistical Public Key Uniformity. As a starting point, we consider FrodoKEM [21], an LWE-based KEM whose public keys and ciphertexts already perfectly pack into bitstrings. We will use the full IND-CCA-secure KEM, since strong pseudorandomness will indirectly require it. FrodoKEM is a *double-LWE* KEM: public keys and ciphertexts are both LWE samples. This technique, from [47], affords the scheme a meaningful improvement in both security guarantees and ciphertext size compared to a Regev-style scheme [54], wherein only one of the aforementioned values is an LWE sample. Unfortunately, double-LWE schemes enjoy only computational public key uniformity, as that uniformity relies on the hardness of LWE.

We construct a KEM with statistical public key uniformity by simply undoing the optimization from [47]. Concretely, rather than defining the public key vector as an LWE sample $\mathbf{b} \leftarrow \mathbf{A}\mathbf{s} + \mathbf{e}$, we define it as the matrix product $\mathbf{b} \leftarrow \mathbf{A}\mathbf{s}$. To retain security, we must modify the dimensions of $\mathbf{A}$ and $\mathbf{s}$. Rather than using a square matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times n}$, we use a rectangular $\mathbf{A} \in \mathbb{Z}_q^{n \times n'}$ where $n' \geq n \log_2 q$. This is necessary to make public keys statistically close to uniform via the leftover hash lemma [38, 47]. We call this construction, after appropriate lifting to the full dimension of the public key and ciphertext, StatFrodoKEM.

Table 2. Comparison of different hybrid PAKE mechanisms at the 128-bit security level. Estimates of runtime are in cycles on an Intel Skylake CPU, and exclude communication time. We also include communication overhead and indicate whether a security proof exists in the adaptive corruption setting. **ParComb** and **SeqComb** are the parallel and sequential combiners defined in [39] and identically in [48]. **X-GA-PAKE** is assumed to use the CTIDH group action [11] and passwords of bitlength $\ell = 80$.

Scheme	Rounds	Runtime (cycles)	Comm. (kB)	Adaptive corruptions
ParComb [X-GA-PAKE, SPAKE2]	1	5.0×10^{10}	20.5	×
CAKE [OEINC [DHKEM-Ell2, StatFrodoKEM]]	2	1.3×10^8	153.8	✓
SeqComb [CAKE [FrodoKEM], SPAKE2]	3	2.3×10^7	19.4	×

Strong Second Pseudorandomness. Recall that OEINC achieves strong second pseudorandomness when outOKEM achieves statistical strong second pseudorandomness, and inOKEM achieves it computationally. The authors of [42] show hashed Diffie–Hellman achieves perfect strong pseudorandomness. Thus it remains only to show that StatFrodoKEM achieves strong second pseudorandomness. Fortunately, the authors of [42] also prove this property for Kyber [8], ML-KEM’s pre-standardization variant. While Kyber and StatFrodoKEM differ on hardness assumptions, they are nearly identical in structure, and, importantly, use the same variant of the Fujisaki-Okamoto transform, hashing the ciphertext into the final output key (something ML-KEM does not do). For these reasons, the arguments of [42, Appendix C] go through without modification.

Performance Tradeoffs. Due to the necessary parameter changes, ciphertexts grow by a factor of about $\log_2 q$. For StatFrodoKEM, this would result in 144kB ciphertexts in the level I parameter set, up from 9.6kB. This would also cause encapsulation and key generation time to increase by a similar factor.

CAKE with hybrid OKEM offers a new set of tradeoffs compared to existing hybrid PAKEs, summarized in Table 2. This construction is two rounds, rather than the three-round sequential combiners and one-round parallel combiners of [39, 48]. It requires less overall computation than the one-round parallel combiner of [48], which relies on X-GA-PAKE, a supersingular isogeny group action PAKE [3] (as well as on a classical PAKE such as SPAKE2 [4]). Inferring from benchmarks on Intel Skylake CPUs [11, 21] and assuming linear scaling for matrix multiplication, we estimate the full runtime of CAKE (ignoring communication costs) to be $384\times$ faster than the one-round alternative. This comes at the cost of an extra communication round and communication overhead of $7.5\times$.

Other Hybrid PAKEs. A similar set of tradeoffs, albeit without security against adaptive corruptions, can be achieved by instantiating other KEM-based PAKEs such as OCAKE [14] and CHIC [7]⁶ with our hybrid OKEM.

⁶ CHIC requires the KEM to be *splittable*, i.e., that all public keys contain a uniform bitstring. ML-KEM is already splittable, since the public matrix seed is a uniform bitstring. In addition, DHKEMs are trivially splittable: we can simply add a sample from $\{0, 1\}^\lambda$ to the end of any DHKEM public key. Since combining splittable

6 Future Work

We briefly describe some unresolved questions, which we leave to future work.

UDP-Compatible Obfuscated Key Exchange. A common payload limit for UDP packets is 1472 bytes. When instantiated with some post-quantum KEMs, the first round of *Drive1* exceeds this threshold. Since IPv4 fragmentation is frequently left disabled due to abuse concerns, there is no obvious way to perform key exchange over UDP while retaining obfuscation properties. Sequence numbers would violate obfuscation requirements, and using any pre-shared information to encrypt data (e.g., *NodeID*) would surrender strong obfuscation. Another challenge is managing retransmission as a result of fragmentation. In addition, stream IDs are necessary for users to be able to *roam*, i.e., maintain a connection while changing their IP or outgoing port. Unfortunately, embedding a stream ID would break obfuscation entirely, and using a symmetric key would either break strong obfuscation, or require the server to do trial decryptions over its set of session keys. It is thus an open problem whether these convenient properties can be recovered in fully encrypted protocols with post-quantum key exchange.

Hybrid ANO-CCA KEMs. OEINC immediately yields a KEM providing hybrid anonymity (ANO-CCA), since SPR-CCA implies ANO-CCA [65]. While SPR-CCA security provides an avenue for proving anonymity, some of the properties we required (in particular, *strong* ciphertext uniformity) arise specifically to enable the proof of SPR-CCA. Thus, it is interesting to ask whether a hybrid ANO-CCA KEM can be constructed with weaker assumptions on the two input KEMs.

Acknowledgements. We thank the anonymous reviewers for their helpful comments. D.S. is supported by Natural Sciences and Engineering Research Council of Canada (NSERC) Discovery grant RGPIN-2022-03187 and NSERC Alliance grant ALLRP 578463-22.

References

1. VMess (2019). <https://www.v2ray.com/en/configuration/protocols/vmess.html>
2. Shadowsocks (2023). <https://shadowsocks.org/doc/what-is-shadowsocks.html>
3. Abdalla, M., Eisenhofer, T., Kiltz, E., Kunzweiler, S., Riepel, D.: Password-authenticated key exchange from group actions. In: Dodis, Y., Shrimpton, T. (eds.) CRYPTO 2022, Part II. LNCS, vol. 13508, pp. 699–728. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-15979-4_24
4. Abdalla, M., Pointcheval, D.: Simple password-based encrypted key exchange protocols. In: Menezes, A. (ed.) CT-RSA 2005. LNCS, vol. 3376, pp. 191–208. Springer, Heidelberg (2005). https://doi.org/10.1007/978-3-540-30574-3_14
5. Anderson, N.: Stakeout: how the FBI tracked and busted a Chicago Anon (2012). <https://arstechnica.com/tech-policy/2012/03/stakeout-how-the-fbi-tracked-and-busted-a-chicago-anon/>

OKEMs yields a splittable OKEM (via concatenation), we can instantiate hybrid CHIC.

6. Angel, Y.: Comment in issue “Improve the OBFS4 obfuscation protocol (#30716)” · The Tor Project / Anti-censorship / Pluggable Transports / lyrebird · GitLab (Aug 2022). https://gitlab.torproject.org/tpo/anti-censorship/pluggable-transports/lyrebird/-/issues/30716#note_2833105
7. Arriaga, A., Barbosa, M., Jarecki, S., Skrobot, M.: C’est très CHIC: a compact password-authenticated key exchange from lattice-based KEM. In: Chung, K.M., Sasaki, Y. (eds.) *Advances in Cryptology - ASIACRYPT 2024*, pp. 3–33. Springer Nature Singapore, Singapore (2024)
8. Avanzi, R., et al.: CRYSTALS-Kyber: algorithm specifications and supporting documentation (version 3.02) (2021). <https://pq-crystals.org/kyber/data/kyber-specification-round3-20210804.pdf>
9. Backendal, M., Bellare, M., Günther, F., Scarlata, M.: When messages are keys: is HMAC a dual-PRF? In: Handschuh, H., Lysyanskaya, A. (eds.) *CRYPTO 2023*, Part III. LNCS, vol. 14083, pp. 661–693. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-38548-3_22
10. Baena, J., Briaud, P., Cabarcas, D., Perlner, R.A., Smith-Tone, D., Verbel, J.A.: Improving support-minors rank attacks: applications to GeMSS and rainbow. In: Dodis, Y., Shrimpton, T. (eds.) *CRYPTO 2022*, Part III. LNCS, vol. 13509, pp. 376–405. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-15982-4_13
11. Banegas, G., et al.: CTIDH: faster constant-time CSIDH. *IACR TCHES* **2021**(4), 351–387 (2021). <https://doi.org/10.46586/tches.v2021.i4.351-387>, <https://tches.iacr.org/index.php/TCHES/article/view/9069>
12. Barbosa, M., Connolly, D., Duarte, J.D., Kaiser, A., Schwabe, P., Varner, K., Westerbaan, B.: X-wing. *CiC* **1**(1), 21 (2024). <https://doi.org/10.62056/a3qj89n4e>
13. Barnes, R., Bhargavan, K., Lipp, B., Wood, C.: Hybrid public key encryption. RFC 9180 (Informational) (2022). <https://doi.org/10.17487/RFC9180>, <https://www.rfc-editor.org/rfc/rfc9180.txt>
14. Beguinet, H., Chevalier, C., Pointcheval, D., Ricosset, T., Rossi, M.: GeT a CAKE: generic transformations from key encapsulation mechanisms to password authenticated key exchanges. In: Tibouchi, M., Wang, X. (eds.) *ACNS 23International Conference on Applied Cryptography and Network Security*, Part II. LNCS, vol. 13906, pp. 516–538. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-33491-7_19
15. Bellare, M., Canetti, R., Krawczyk, H.: Keying hash functions for message authentication. In: Koblitz, N. (ed.) *CRYPTO 1996*. LNCS, vol. 1109, pp. 1–15. Springer, Heidelberg (1996). https://doi.org/10.1007/3-540-68697-5_1
16. Bellare, M., Rogaway, P.: Entity authentication and key distribution. In: Stinson, D.R. (ed.) *CRYPTO 1993*. LNCS, vol. 773, pp. 232–249. Springer, Heidelberg (1994). https://doi.org/10.1007/3-540-48329-2_21
17. Bernstein, D.J., et al.: KyberSlash: Exploiting secret-dependent division timings in kyber implementations. *Cryptology ePrint Archive*, Report 2024/1049 (2024). <https://eprint.iacr.org/2024/1049>
18. Bernstein, D.J., Hamburg, M., Krasnova, A., Lange, T.: Elligator: elliptic-curve points indistinguishable from uniform random strings. In: Sadeghi, A.R., Gligor, V.D., Yung, M. (eds.) *ACM CCS 2013*, pp. 967–980. ACM Press (2013). <https://doi.org/10.1145/2508859.2516734>
19. Beullens, W.: Breaking rainbow takes a weekend on a laptop. In: Dodis, Y., Shrimpton, T. (eds.) *CRYPTO 2022*, Part II. LNCS, vol. 13508, pp. 464–479. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-15979-4_16

20. Bindel, N., Brendel, J., Fischlin, M., Goncalves, B., Stebila, D.: Hybrid key encapsulation mechanisms and authenticated key exchange. In: Ding, J., Steinwandt, R. (eds.) *Post-Quantum Cryptography - 10th International Conference, PQCrypto 2019*, pp. 206–226. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-25510-7_12
21. Bos, J.W., et al.: Frodo: Take off the ring! Practical, quantum-secure key exchange from LWE. In: Weippl, E.R., Katzenbeisser, S., Kruegel, C., Myers, A.C., Halevi, S. (eds.) *ACM CCS 2016*, pp. 1006–1018. ACM Press (2016). <https://doi.org/10.1145/2976749.2978425>
22. Canetti, R.: Universally composable security: a new paradigm for cryptographic protocols. In: *42nd FOCS*, pp. 136–145. IEEE Computer Society Press (2001). <https://doi.org/10.1109/SFCS.2001.959888>
23. Castryck, W., Decru, T.: An efficient key recovery attack on SIDH. In: Hazay, C., Stam, M. (eds.) *EUROCRYPT 2023, Part V*. LNCS, vol. 14008, pp. 423–447. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-30589-4_15
24. Connolly, D.: Hybrid PQ/T key encapsulation mechanisms. Internet draft draft-irtf-cfrg-hybrid-kems-03, Internet Engineering Task Force (2025). <https://datatracker.ietf.org/doc/draft-irtf-cfrg-hybrid-kems-03>
25. D’Anvers, J.-P., Karmakar, A., Sinha Roy, S., Vercauteren, F.: Saber: module-LWR based key exchange, CPA-secure encryption and CCA-secure KEM. In: Joux, A., Nitaj, A., Rachidi, T. (eds.) *AFRICACRYPT 2018*. LNCS, vol. 10831, pp. 282–305. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-89339-6_16
26. Donenfeld, J.A.: WireGuard: next generation kernel network tunnel. In: *NDSS 2017*. The Internet Society (2017). <https://doi.org/10.14722/ndss.2017.23160>
27. Fenske, E., Johnson, A.: Bytes to schlep? Use a FEP: Hiding protocol metadata with fully encrypted protocols. In: *ACM CCS 2024*, pp. 1982–1996. CCS 2024, Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3658644.3690198>
28. French cybersecurity agency: ANSSI views on the post-quantum cryptography transition (2022). <https://cyber.gouv.fr/sites/default/files/document/EN-Position.pdf>
29. Gajland, P., Hwang, V., Janneck, J.: ShadowFax: a deniability-preserving AKEM combiner. *Cryptology ePrint Archive, Paper 2025/154* (2025). <https://eprint.iacr.org/2025/154>
30. German federal office for information security: BSI TR-02102-1, cryptographic mechanisms: recommendations and key lengths (2024). version 2024-01, <https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Publications/TechGuidelines/TG02102/BSI-TR-02102-1.pdf>
31. Giacon, F., Heuer, F., Poettering, B.: KEM combiners. In: Abdalla, M., Dahab, R. (eds.) *PKC 2018*. LNCS, vol. 10769, pp. 190–218. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-76578-5_7
32. Grubbs, P., Maram, V., Paterson, K.G.: Anonymous, robust post-quantum public key encryption. In: Dunkelman, O., Dziembowski, S. (eds.) *EUROCRYPT 2022, Part III*. LNCS, vol. 13277, pp. 402–432. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-07082-2_15
33. Günther, F., Rosenberg, M., Stebila, D., Veitch, S.: Hybrid obfuscated key exchange and KEMs. *Cryptology ePrint Archive, Paper 2025/408* (2025). <https://eprint.iacr.org/2025/408>

34. Günther, F., Stebila, D., Veitch, S.: Obfuscated key exchange. In: ACM CCS 2024, pp. 2385–2399. CCS 2024, Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3658644.3690220>
35. Guo, Q., Hlauschek, C., Johansson, T., Lahr, N., Nilsson, A., Schröder, R.L.: Don't reject this: key-recovery timing attacks due to rejection-sampling in HQC and BIKE. *IACR TCHES* **2022**(3), 223–263 (2022). <https://doi.org/10.46586/tches.v2022.i3.223-263>
36. Guo, Q., Johansson, T., Nilsson, A.: A key-recovery timing attack on post-quantum primitives using the Fujisaki-Okamoto transformation and its application on FrodoKEM. In: Micciancio, D., Ristenpart, T. (eds.) CRYPTO 2020. LNCS, vol. 12171, pp. 359–386. Springer, Cham (2020). <https://doi.org/10.1007/978-3-030-56880-1.13>
37. Hall, D.: Zero trust WARP: tunneling with a MASQUE (2024). <https://blog.cloudflare.com/zero-trust-warp-with-a-masque>
38. Håstad, J., Impagliazzo, R., Levin, L.A., Luby, M.: A pseudorandom generator from any one-way function. *SIAM J. Comput.* **28**(4), 1364–1396 (1999)
39. Hesse, J., Rosenberg, M.: PAKE combiners and efficient post-quantum instantiations. *Cryptology ePrint Archive*, Paper 2024/1621 (2024). <https://eprint.iacr.org/2024/1621>
40. Huang, S., Sim, R.Q., Chuengsatiansup, C., Guo, Q., Johansson, T.: Cache-timing attack against HQC. *IACR TCHES* **2023**(3), 136–163 (2023). <https://doi.org/10.46586/tches.v2023.i3.136-163>
41. Iyengar (Ed.), J., Thomson (Ed.), M.: QUIC: A UDP-based multiplexed and secure transport. RFC 9000 (Proposed Standard) (2021). <https://doi.org/10.17487/RFC9000>, <https://www.rfc-editor.org/rfc/rfc9000.txt>
42. Januzelli, J., Roy, L., Xu, J.: Under what conditions is encrypted key exchange actually secure? In: Fehr, S., Fouque, P.A. (eds.) *Advances in Cryptology - EURO-CRYPT 2025*, pp. 451–481. Springer Nature Switzerland, Cham (2025)
43. Krawczyk, H., Bellare, M., Canetti, R.: HMAC: Keyed-hashing for message authentication. RFC 2104 (Informational) (1997). <https://doi.org/10.17487/RFC2104>, <https://www.rfc-editor.org/rfc/rfc2104.txt>, updated by RFC 6151
44. Krawczyk, H., Eronen, P.: HMAC-based extract-and-expand key derivation function (HKDF). RFC 5869 (Informational) (2010). <https://doi.org/10.17487/RFC5869>, <https://www.rfc-editor.org/rfc/rfc5869.txt>
45. Krawczyk, H.: Cryptographic extraction and key derivation: the HKDF scheme. In: Rabin, T. (ed.) CRYPTO 2010. LNCS, vol. 6223, pp. 631–648. Springer, Heidelberg (2010). <https://doi.org/10.1007/978-3-642-14623-7.34>
46. Langley, A., Hamburg, M., Turner, S.: Elliptic curves for security. RFC 7748 (Informational) (2016). <https://doi.org/10.17487/RFC7748>, <https://www.rfc-editor.org/rfc/rfc7748.txt>
47. Lindner, R., Peikert, C.: Better key sizes (and attacks) for LWE-based encryption. In: Kiayias, A. (ed.) CT-RSA 2011. LNCS, vol. 6558, pp. 319–339. Springer, Heidelberg (2011). <https://doi.org/10.1007/978-3-642-19074-2.21>
48. Lyu, Y., Liu, S.: Hybrid password authentication key exchange in the UC framework. *Cryptology ePrint Archive*, Paper 2024/1630 (2024). <https://eprint.iacr.org/2024/1630>
49. Maram, V., Xagawa, K.: Post-quantum anonymity of Kyber. In: Boldyreva, A., Kolesnikov, V. (eds.) PKC 2023, Part I. LNCS, vol. 13940, pp. 3–35. Springer, Cham (2023). <https://doi.org/10.1007/978-3-031-31368-4.1>
50. Mathewson, N.: NTOR protocol with extra data, version 3. <https://spec.torproject.org/proposals/332-ntor-v3-with-extra-data.html> (2021)

51. Naehrig, M., et al.: FrodoKEM. Tech. rep., National institute of standards and technology (2020). <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>
52. NIST: Digital signature standard (DSS) (2023). FIPS 186-5. <https://doi.org/10.6028/NIST.FIPS.186-5>
53. Radar, C.: Adoption & Usage Worldwide — Cloudflare Radar. <https://radar.cloudflare.com/adoption-and-usage#post-quantum-encryption-adoption>
54. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. In: Gabow, H.N., Fagin, R. (eds.) 37th ACM STOC, pp. 84–93. ACM Press (2005). <https://doi.org/10.1145/1060590.1060603>
55. Rescorla, E., Oku, K., Sullivan, N., Wood, C.A.: TLS encrypted client hello. Internet draft draft-ietf-tls-esni-22, Internet engineering task force (2024). <https://datatracker.ietf.org/doc/draft-ietf-tls-esni-22>
56. Santos, B.F.D., Gu, Y., Jarecki, S.: Randomized half-ideal cipher on groups with applications to UC (a)PAKE. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023, Part V. LNCS, vol. 14008, pp. 128–156. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-30589-4_5
57. Schwartz, B.M., Patton, C.: The Pseudorandom Extension for cTLS. Internet Draft draft-cpbs-pseudorandom-ctls-01, Internet engineering task force (2022). <https://datatracker.ietf.org/doc/draft-cpbs-pseudorandom-ctls-01>
58. The tor project: obfs4 (the obfourscator), protocol specification, version c0898c2d (2019). <https://gitlab.torproject.org/tpo/anti-censorship/pluggable-transport/lyrebird/-/blob/main/doc/obfs4-spec.txt>
59. Tibouchi, M.: Elligator squared: uniform points on elliptic curves of prime order as uniform random strings. In: Christin, N., Safavi-Naini, R. (eds.) FC 2014. LNCS, vol. 8437, pp. 139–156. Springer, Heidelberg (2014). https://doi.org/10.1007/978-3-662-45472-5_10
60. Wails, R., Jansen, R., Johnson, A., Sherr, M.: Proteus: programmable protocols for censorship circumvention. In: Free and Open Communication on the Internet, pp. 50–66. No. 2 (2023)
61. Westerbaan, B.: The state of the post-quantum Internet (2024). <https://blog.cloudflare.com/pq-2024>
62. Westerbaan, B., Wood, C.A.: X25519Kyber768Draft00 hybrid post-quantum KEM for HPKE. Internet Draft draft-westerbaan-cfrg-hpke-xyber768d00-03, Internet engineering task force (2024). <https://datatracker.ietf.org/doc/draft-westerbaan-cfrg-hpke-xyber768d00>, p. 20
63. White, A.M., Matthews, A.R., Snow, K.Z., Monroe, F.: Phonotactic reconstruction of encrypted VoIP conversations: Hookt on Fon-iks. In: 2011 IEEE Symposium on Security and Privacy, pp. 3–18. IEEE Computer Society Press (2011). <https://doi.org/10.1109/SP.2011.34>
64. Wu, M., et al.: How the great firewall of china detects and blocks fully encrypted traffic. In: Calandrino, J.A., Troncoso, C. (eds.) USENIX Security 2023, pp. 2653–2670. USENIX Association (2023)
65. Xagawa, K.: Anonymity of NIST PQC round 3 KEMs. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022, Part III. LNCS, vol. 13277, pp. 551–581. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-07082-2_20
66. Xue, D., Ablove, A., Ramesh, R., Danciu, G.K., Ensafi, R.: Bridging barriers: a survey of challenges and priorities in the censorship circumvention landscape. In: Balzarotti, D., Xu, W. (eds.) USENIX Security 2024. USENIX Association (2024)